

Dr. D Y PATIL TECHNICAL CAMPUS
AJEENKYA DY PATIL SCHOOL OF ENGINEERING

Dr. D Y Patil Knowledge City, Charholi Bk, Via Lohegaon, Pune 412 081



A PROJECT REPORT ON

**eSC-V: Machine-Mode CFI-Hardened RISC-V SoC
in VHDL**

Submitted to the Savitribai Phule Pune University

In partial fulfillment of the requirement for the award of the Degree of

**BACHELOR OF ENGINEERING IN
ELECTRONICS AND TELECOMMUNICATION ENGINEERING**

by

1. Arjun Salunkhe	Exam. Seat No. B400880228
2. Atharva Bhosale	Exam. Seat No. B400880157
3. Sourabh Bansode	Exam. Seat No. B400880154
4. Srushti Shinde	Exam. Seat No. B400880233

Under the guidance of

Dr. K. S. Vaidya

Department of Electronics and Telecommunication Engineering

AJEENKYA DY PATIL SCHOOL OF ENGINEERING

A.Y. 2025-26

Dr. D Y PATIL TECHNICAL CAMPUS
AJEENKYA DY PATIL SCHOOL OF ENGINEERING

Dr. D Y Patil Knowledge City, Charholi Bk, Via Lohegaon, Pune 412 081



CERTIFICATE

This is to certify that Mr. Arjun Salunkhe - Seat No. B400880228
 Mr. Atharva Bhosale - Seat No. B400880157
 Mr. Sourabh Bansode - Seat No. B400880154
 Ms. Srushti Shinde - Seat No. B400880233

have satisfactorily completed Project Part-II and presented a report on the topic titled **eSC-V: Machine-Mode CFI-Hardened RISC-V SoC in VHDL** at Ajeenkya DY Patil School of Engineering in partial fulfillment of requirement of Savitribai Phule Pune University, Pune, for the degree of BE (Electronics and Telecommunication Engineering) in semester VIII during academic year 2025-2026.

Dr. K. S. Vaidya
Project Guide

Dr. Sanjay Koli
& **Dr. Saniya Ansari**
Project Coordinator

Dr. K. S. Vaidya
H.O.D E&TC

Dr. F. B. Sayyad
Principal

Examiner Name: _____

Sign: _____

INDEX			
CHAPTER NO.	SUB NO.	TITLE	PAGE NO.
1		INTRODUCTION	2
	1.1	Project Overview	3
	1.2	Project Scope	4
2		LITERATURE SURVEY	5
	2.1	Related Works	6
	2.2	Comparative Analysis	7
	2.3	Research Gap	8
	2.4	Problem Statement	9
3		ARCHITECTURE	11
	3.1	RISC-V Architecture	12
	3.2	RISC-V Execution Model	13
	3.3	RISC-V Encodings	16
	3.4	Base Instruction Set	18
	3.5	RISC-V Ratified Extentions	21
	3.6	RISC-V Draft Extentions	24
4		HARDWARE DESIGN	27
	4.1	System-On-Chip (SoC) Structure	28
	4.2	CPU Core	31
	4.3	Unified Memory Controller & BRAM Interface	50
	4.4	UART Peripheral	51
5		SOFTWARE DESIGN	53
	5.1	Software Overview	54
	5.2	Bare-metal Bootstrapping & Memory Layout	54
	5.3	C Standard Library Integration & System Call Interface	57

	5.4	Hardware Peripheral & Debug Drivers	59
	5.5	Ported Applications & Validation Suite	60
	5.6	Web-Based SoC Demonstration Interface	61
6		METHODOLOGY	64
	6.1	Methodology Overview	65
	6.2	Control-Flow Hijacking Mechanics	65
	6.3	Hardware-Enforced Mitigations	67
7		REQUIREMENTS	69
	7.1	Requirements Overview	70
	7.2	Hardware Requirements	71
	7.3	Software Requirements	73
8		VERIFICATION	76
	8.1	Overview of the RISCOF Framework	77
	8.2	Hardware Verification Testbench & SoC Debug Port Integration	77
	8.3	Custom Python Plugin & Test Orchestration	77
	8.4	Verification Coverage & Compliance Results	78
9		RESULTS	79
	9.1	Synthesis and Performance Overhead Results	80
10		CONCLUSION	82
	10.1	Conclusion	83
11		REFERENCES	85
	11.1	References	86
12		PROJECT ACHIVEMENTS	88
	12.1	Peer-Reviewed Publications & Research Contributions	89
	12.2	NLPC 2026 Organised by IETE	90
	12.3	Sponsorship	90

LIST OF TABLES

Table No.	Title	Page No.
1	RISC-V Base Instruction Encoding Formats	17
2	RISC-V Immediate Generation Mapping	18
3	RV32I Base Instruction Set Bit Allocations	19
4	Zicsr Instruction Set Bit Allocations	21
5	Zicfilp Landing Pad Instruction Encoding	24
6	Smpmpind Indirect PMP Register Target Allocations	24
7	Zicfiss Shadow Stack Instruction Primitive Summary	26

LIST OF FIGURES

Figure No.	Title	Page No.
1	eSC-V SoC Block Diagram	28
2	eSC-V detailed Block Diagram	31
3	Web-Based SoC Demonstration Interface Home Page.	62
4	Interactive ASCII-Tetris Game Layout with Live Architectural Metrics.	62
5	Full-Duplex Pong-C Application Dashboard over Serial Stream.	62
6	Unsafe CFI Simulation (Hardware Protections Disabled — Attack Vector Successfully Exploited)	63
7	Safe CFI Simulation (Hardware Protections Enabled — Exploit Defeated via Precise System Trap)	63
8	Sipeed Tang Primer 20k Diagram	71
9	ESP-PROG Diagram	72
10	Nix Flakes	73
11	Results Graph	80
12	Participation certificates from the National Level Project Competition (NLPC) 2026 organised by IETE	90
13	Sponsorship letter	90

ABSTRACT

The **eSC-V** project introduces a custom 32-bit RISC-V System-on-Chip (SoC) engineered entirely from scratch in VHDL. At its foundation is a **5-stage pipeline**: Instruction Fetch, Decode, Execute, Memory, and Writeback. To maintain execution efficiency without unnecessary latency, the microarchitecture integrates a Hazard Detection Unit that manages single-cycle stalls for load-use data dependencies, alongside a comprehensive Forwarding Circuitry that resolves dynamic data hazards across the pipeline stages. The core is integrated with a unified, dual-port memory controller mapped to FPGA Block RAM (BRAM) and a memory-mapped UART transceiver. The design underwent formal architectural verification using the RISCOF framework against the Sail and Spike models, achieving 100% compliance on applicable base tests. The complete processor was synthesized using an end-to-end open-source EDA toolchain and deployed on a Sipeed Tang Primer 20K FPGA running at 54 MHz.

Because eSC-V targets microcontroller-class applications, it operates purely in a bare-metal Machine-mode environment without an Operating System or Memory Management Unit (MMU). To overcome the difficulties of fragmented hardware toolchains, the project uses Nix Flakes to provision a fully reproducible GNU GCC compiler environment. Software execution begins with a custom `crt0.S` assembly bootstrap that configures the C runtime, stack, and security extensions prior to entering the main application. The platform establishes a standard C library (`libc`) environment by integrating Newlib, utilizing a custom bare-metal interface (`syscalls.c`) to implement essential system calls like `_sbrk`, `_read`, and `_write`. This software stack is robust enough to compile and run interactive C applications, such as ASCII Tetris and Pong, directly over the serial connection.

The overarching objective of the SoC is to protect these bare-metal environments from complex **Code-Reuse Attacks (CRAs)**, specifically Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP). To neutralize JOP, the hardware integrates the ratified RISC-V **Zicfilp** extension, enforcing forward-edge Control-Flow Integrity (CFI) by requiring indirect jumps to target valid landing pad (`lpad`) instructions. To defeat ROP, eSC-V implements the draft **Smcfiss** and **Smpmpind** extensions to enforce a backward-edge shadow stack. Because bare-metal environments lack virtual memory isolation, an inline Physical Memory Protection (PMP) unit acts as a hardware filter, isolating the shadow stack from unauthorized memory stores. Any control-flow violation triggers a custom two-stage pipeline flush, safely trapping the system while requiring an exceptionally low 6.68% logic overhead footprint.

CHAPTER 1

INTRODUCTION

1.1 Project Overview

eSC-V is an open-source, 5-stage pipelined RISC-V System-on-Chip (SoC) implemented entirely in VHDL. The project implements the RV32I base integer instruction set alongside the Zicsr, Zicfilp, Smcfiss, and Smpmpind extensions. The primary objective of this project is to deliver robust, hardware-enforced Control-Flow Integrity (CFI) specifically tailored for bare-metal, Machine-mode (M-mode) microcontrollers. By embedding security primitives directly into the silicon logic, eSC-V provides a highly secure execution environment capable of thwarting modern code-reuse attacks.

To achieve this, the architecture natively integrates specialized RISC-V security extensions. It defends against Jump-Oriented Programming (JOP) attacks using the Zicfilp extension, which enforces forward-edge protection through hardware landing pads. Furthermore, it protects against Return-Oriented Programming (ROP) attacks by implementing the draft Smcfiss and Smpmpind extensions, which enforce backward-edge protection via a hardware-backed shadow stack.

Architecturally, the SoC is designed with a dual-port unified memory controller that utilizes Harvard architecture semantics over a single underlying memory array, which synthesizes efficiently to Block RAM (BRAM). The system also features a built-in UART transceiver for serial communication and a Physical Memory Protection (PMP) unit. The hardware design is highly practical and has been successfully synthesized for the Tang Primer 20K FPGA, running at 54 MHz using an open-source bitstream.

Beyond the hardware itself, the eSC-V project provides a complete and reproducible full-stack development environment. Leveraging the Nix package manager, the project guarantees a consistent build environment, including a custom GCC toolchain required to compile the advanced CFI extensions.

The system's reliability and standard compliance have been rigorously tested. The CPU core has been verified against the Sail and Spike formal models using the RISC-V Compatibility Framework (RISCOF). To demonstrate its real-world viability, the SoC successfully bootstraps standard C libraries and runs complex interactive applications, including ASCII Tetris and Pong. Finally, the project includes a purpose-built security validation suite with proof-of-concept exploits, empirically proving that when CFI protections are enabled, the hardware successfully detects and traps both ROP and JOP violations before arbitrary code execution can occur.

1.2 Project Scope

The scope of the eSC-V project encompasses the end-to-end design, implementation, and verification of a secure microcontroller system. Operating strictly within a bare-metal, Machine-mode execution environment, the project demonstrates how fundamental security primitives can be efficiently embedded directly into a compact hardware footprint. Furthermore, a primary emphasis is placed on maintaining an entirely open-source ecosystem; by avoiding proprietary software dependencies, the project establishes a transparent, highly accessible, and reproducible platform for development and architectural research.

The primary features and deliverables of the project include:

- **Core Architecture:** A fully functional 5-stage pipelined RV32I processor implemented in VHDL, complete with hazard detection, data forwarding, and a dual-port unified memory controller.
- **Security Extensions:** Hardware integration of the Zicsr, Zicfilp (Landing Pads), Smcfiss (Shadow Stack), and Smpmpind extensions to enforce strict Control-Flow Integrity (CFI).
- **Peripherals:** A built-in UART transceiver for serial communication.
- **Software Ecosystem:** A customized build system capable of bootstrapping standard C and newlib (libc) functions.
- **Demonstration Applications:** Real-world software validation using interactive games, specifically ASCII Tetris and Pong, executed over UART. A demonstration website for easy demonstration of the games and security features.
- **Verification:** Architectural compliance testing against the Sail and Spike formal models using the RISC-V Compatibility Framework (RISCOF), alongside physical validation on a Tang Primer 20K FPGA.

Out of Scope: To maintain focus on the core CFI security objectives, certain advanced System-on-Chip features were intentionally excluded from this iteration. Components such as a bootloader, Wishbone interconnect, branch prediction unit, hardware caches, and DDR3 memory controllers are considered out of scope for the current design, though they provide a clear roadmap for future expansion.

CHAPTER 2

LITERATURE SURVEY

2.1 Related Works

The problem of securing bare-metal embedded systems from code-reuse attacks has attracted considerable research attention over the past decade. The rapid expansion of the Internet-of-Things (IoT) and advanced robotics has shifted highly critical control tasks to resource-constrained microcontrollers. Because these systems have historically operated in isolated environments, architectural security has not kept pace with the evolving threat landscape, leaving them highly vulnerable to sophisticated exploits. Furthermore, because the vast majority of embedded firmware is still developed using memory-unsafe languages such as C and C++, the risk of memory corruption vulnerabilities remains pervasive.

Early mitigation strategies attempted to port operating system-level defenses to the embedded domain. However, researchers quickly identified that software-only defenses are severely insufficient for real-time microcontroller environments. The necessity of real-time monitoring without degrading deterministic execution has driven the exploration of numerous hardware-assisted architectures, each contributing unique mechanisms to isolate and monitor control flows without relying on an underlying Operating System.

SOFIA (2016): De Clercq et al. introduced SOFIA (Software and Control Flow Integrity Architecture), a foundational work that enforced CFI through custom hardware monitors explicitly designed for safety-critical systems. SOFIA demonstrated the immediate performance advantages of hardware enforcement by entirely offloading CFI checks from the software stack. The proposed architecture was implemented on the open-source LEON3 soft microprocessor. Key findings from this implementation showed that while software instrumentation caused unacceptable delays, the hardware approach still carried a substantial area overhead of 28.2% and an execution cycle overhead of 13.7%. While SOFIA demonstrated robust security against code-reuse attacks, its fundamental reliance on proprietary instruction set modifications limited its portability, making it incompatible with standard upstream compiler ecosystems like GCC and LLVM.

FIXER (2019): De et al. advanced the domain by proposing FIXER, a Flow Integrity Extensions architecture specifically targeting embedded RISC-V platforms. FIXER introduced dedicated monitor hardware capable of validating Control-Flow Graphs (CFGs) at runtime without requiring deep modifications to the core CPU pipeline. Instead, it leveraged the integrated Rocket Custom Coprocessor (RoCC) interface on a RISC-V RocketChip SoC. Findings demonstrated highly efficient protection, reducing energy overhead by 60% compared to software alternatives, while adding only a 1.5% execution time overhead and a remarkably low 2.9% area overhead. While FIXER was highly effective and resource-efficient, it relied heavily on non-standard hardware extensions and specialized coprocessor interfaces. This reliance created ecosystem fragmentation, as custom toolchains were required to generate the specific metadata FIXER needed to function, hindering widespread adoption.

FPGA-Based CFI Coprocessors (2020): Maunero et al. explored a dedicated FPGA coprocessor approach to enforce CFI for bare-metal embedded systems executing directly from Flash memory. This work highlighted a critical constraint in embedded systems: the timing jitter introduced by software instrumentation is unacceptable for deterministic, deadline-sensitive real-time workloads. The findings demonstrated that by monitoring the CFG using an external FPGA connected to the CPU bus, it is possible to protect the execution context from corruption even when entering highly unpredictable code paths, such as Interrupt Service Routines (ISRs). However, the external coprocessor model inherently introduces additional silicon area, power consumption, and bus-interface latency, making it less ideal for deeply embedded, low-power microcontrollers.

Real-Time CFI for Embedded Systems (2019): Walls et al. investigated the temporal impact of CFI enforcement in real-time embedded systems, introducing a defense framework called RECFISH. Their findings confirmed that hardware-assisted approaches can achieve near-zero timing overhead, while traditional software-based methods introduce unacceptable latency jitter in applications like robotics and industrial control systems. Notably, RECFISH bypassed the need for custom compiler toolchains by retrofitting precompiled ARM binaries with CFI protections. Furthermore, it implemented a memory-isolation approach for ARM that functioned without a Memory Management Unit (MMU), adapting the FreeRTOS real-time operating system. While innovative, binary rewriting is complex to audit and maintain compared to natively compiled security policies.

CVA6-CFI (2026): With the recent ratification of official RISC-V security extensions, Manoni et al. presented CVA6-CFI, representing the first major implementation of the RISC-V Zicfilp and Zicfiss extensions. CVA6-CFI demonstrated that these standards-based CFI extensions can be successfully integrated into a complex, superscalar, out-of-order processor pipeline. The findings confirmed the functional validity of these new ISA extensions in providing robust security against Control-Reuse Attacks. However, a critical limitation remains regarding its deployment domain: the CVA6 targets Linux-capable application processors. It fundamentally relies on full Memory Management Unit (MMU) support and page-based memory attributes to isolate the shadow stack across User and Supervisor privilege levels. Consequently, this architecture is fundamentally unsuited to resource-constrained bare-metal Machine-mode environments, which lack virtual memory isolation entirely.

2.2 Comparative Analysis

A comparative analysis of the aforementioned hardware-assisted CFI approaches reveals distinct commonalities in their objectives, alongside highly contrasting methodologies regarding implementation and standardisation.

- **Commonalities in Threat Mitigation:** All evaluated works inherently recognize the failure of pure software-based CFI in real-time environments. Software mitigations in-

evitably introduce timing jitter and latency overheads that violate the strict deadlines of deterministic workloads. SOFIA, FIXER, and RECFISH all acknowledge that dedicated hardware evaluation is strictly required to execute fine-grained CFG checks without delaying the primary execution pipeline. Furthermore, they share a common goal of providing forward and backward-edge protection independent of an MMU, validating hardware-based memory isolation as the only viable replacement for virtual-memory-backed protection in embedded domains.

- **Contrasts in Pipeline Integration:** The primary divergence among these systems lies in how the security policies are injected into the CPU pipeline. SOFIA directly modified the processor core (LEON3), which provided tight integration but created high area overhead (28.2%). In contrast, FIXER and the FPGA-based coprocessor models externalized the monitoring logic to dedicated coprocessors. This preserved the integrity of the base CPU IP and greatly reduced core area overhead (FIXER achieved 2.9%), but introduced bus latencies. Meanwhile, RECFISH utilized binary rewriting on ARM, completely decoupling the hardware enforcement from the compiler toolchain but complicating the deployment pipeline.
- **Contrasts in Ecosystem Standardization:** The most significant contrast is between older frameworks and recent RISC-V developments. SOFIA, FIXER, and RECFISH all relied on proprietary metadata formats, custom instruction sets, or custom binary patches. This reliance forces developers to maintain custom forks of compilers (like GCC) to support the hardware. Conversely, CVA6-CFI and the proposed eSC-V architecture adopt newly ratified, standardized RISC-V ISA extensions (Zicfilp, Zicfiss, and Smcfiss). This ensures native, long-term compatibility with mainline open-source toolchains like GCC and LLVM without requiring custom binary retrofitting or proprietary vendor lock-in.

2.3 Research Gap

The existing body of work reveals a clear and persistent gap in the domain of embedded security. Prior hardware CFI architectures either rely heavily on proprietary, non-standardized instruction set extensions and external coprocessor interfaces (as seen in SOFIA, FIXER, and FPGA monitors), or they target application-class processors equipped with full virtual memory support (as seen in CVA6-CFI).

To date, no prior open-source work has demonstrated the implementation of the ratified RISC-V Zicfilp extension alongside the draft Smcfiss and Smpmpind extensions directly inside a minimalist, bare-metal Machine-mode core. Because bare-metal cores lack an MMU, standard page-based shadow stack isolation cannot be used. The eSC-V project directly addresses this critical gap by proving that active M-mode security, utilizing indirect CSR access (via *misselect* and *mireg*) and Physical Memory Protection (PMP) to isolate the shadow stack, can be

achieved. Furthermore, eSC-V aims to demonstrate that this standard-compliant protection can be implemented with a highly efficient logic overhead (approximately 6.68%) and zero reliance on custom compiler forks.

2.4 Problem Statement

The rapid proliferation of IoT devices, smart industrial sensors, and autonomous robotics platforms has exponentially increased the deployment of bare-metal microcontrollers in safety-critical applications. These systems operate under strict architectural constraints: they function entirely in Machine-mode (M-mode) without the safety net of an operating system or a Memory Management Unit (MMU). Despite operating in such highly sensitive environments, the firmware driving these devices is predominantly written in memory-unsafe languages such as C and C++.

This combination of memory-unsafe programming and exposed hardware architectures leaves bare-metal systems critically vulnerable to Code-Reuse Attacks (CRAs). By exploiting simple memory corruption bugs, adversaries can execute arbitrary code without needing to inject new malicious executables. This architectural reality exposes bare-metal systems to two principal classes of control-flow attacks:

- **Return-Oriented Programming (ROP):** In an ROP attack, an adversary exploits a buffer overflow vulnerability to corrupt the stored return addresses located on the standard data stack. When the vulnerable function executes its epilogue and returns, it pops the corrupted address, redirecting the processor's execution flow. The attacker sequentially chains together existing, benign firmware instructions (known as "gadgets") to incrementally hijack control flow and ultimately achieve arbitrary, malicious code execution.
- **Jump-Oriented Programming (JOP):** JOP attacks target the forward-edge of the control flow. Instead of targeting the stack, an attacker corrupts indirect targets stored in memory or registers, such as function pointers or virtual table dispatchers. When an indirect jump instruction is executed, the processor is redirected to an unauthorized code location, allowing the attacker to bypass standard procedural logic.

The Inadequacy of Existing Defenses

In high-performance application environments, these threats are heavily mitigated using operating system-level defenses such as Address Space Layout Randomisation (ASLR) and virtual memory isolation. Bare-metal microcontrollers, however, operate in a single, flat physical address space.

While software-based Control-Flow Integrity (CFI) can theoretically insert runtime validation checks via compiler instrumentation, the resulting instruction execution overhead introduces unacceptable latency and timing jitter. For deterministic, real-time workloads (such as motor control in robotics or grid synchronization), these software-induced delays can cause catastrophic physical failures.

Furthermore, because M-mode systems fundamentally lack privilege separation, the stakes of a compromise are absolute. A single successful control-flow hijack grants an attacker unrestricted, root-level access to all physical memory, direct memory access (DMA) controllers, memory-mapped peripherals, and critical system resources. Existing hardware-assisted CFI proposals have attempted to solve this, but they either rely on proprietary instruction set extensions that lack widespread compiler support, or they falsely assume the availability of virtual memory infrastructure (MMUs) that simply does not exist in bare-metal environments.

Core Research Question

The core problem addressed by the eSC-V project can therefore be defined as follows:

How can standards-compliant, hardware-enforced Control-Flow Integrity be feasibly implemented within a resource-constrained, bare-metal RISC-V Machine-mode processor, providing robust, deterministic protection against both ROP and JOP attacks with negligible logic overhead and full open-source toolchain reproducibility?

CHAPTER 3

ARCHITECTURE

3.1 RISC-V Architecture

An Instruction Set Architecture (ISA) defines the abstract interface between hardware and software, specifying instruction encodings, data types, register states, memory addressing modes, and control flow mechanisms. Microprocessors have historically relied on proprietary ISAs, such as x86 for desktop/server domains and ARM for embedded systems. RISC-V is an open-standard, royalty-free ISA based on Reduced Instruction Set Computer (RISC) principles, maintained by RISC-V International. Unlike commercial processor designs, RISC-V is a set of open specifications rather than a physical microarchitectural implementation or an open-source hardware core. This permits any entity to develop, manufacture, and commercialize RISC-V-compliant processors without licensing fees or legal dependencies on proprietary intellectual property.

3.1.1 Core Philosophy

The architectural structure of RISC-V relies on a strict delineation between a mandatory base integer ISA and optional modular extensions. This design prevents architectural bloat, which occurs in legacy ISAs when succeeding hardware generations retain outdated instructions solely to maintain backward software compatibility. By freezing the base ISA and isolating specialized functions into discrete modules, RISC-V isolates decoder complexity.

The base integer ISA establishes the fundamental execution environment, register files, and data types required by a standard compiler toolchain. For 32-bit systems, the RV32I variant represents the standard base configuration, defining approximately 40 integer instructions. These operations encompass basic arithmetic, logic, shifts, memory synchronization, and conditional or unconditional branch execution, operating over 32 general-purpose architectural registers.

Standardized extensions are appended to the base ISA to support specific computational workloads:

- **M (Multiply):** Hardware integer multiplication and division instructions.
- **A (Atomic):** Read-modify-write memory operations utilized for multi-core synchronization and thread primitives.
- **F/D (Floating-Point):** Single- and double-precision hardware floating-point instruction execution.
- **C (Compressed):** 16-bit instructions designed to optimize code density and instruction cache utilization.
- **Zicsr (Control and Status Registers):** Subsystems governing privilege levels, exception handling, hardware counters, and trap vector registers.

The specification reserves designated opcode spaces for custom, application-specific instructions. This structural attribute allows the integration of domain-specific accelerators or custom hardware monitors directly into the core instruction stream without interfering with standard compiler infrastructure.

3.1.2 Technical Justification and Utilization

The utilization of RISC-V within embedded, industrial, and specialized research environments is driven by quantifiable technical and economic factors:

1. **Mitigation of Vendor Lock-in:** Utilizing an open standard ensures software portability across different hardware providers. If a specific silicon implementation is discontinued or compromised, the underlying software stack can be remap-compiled onto an alternative RISC-V core or implemented natively in a custom Hardware Description Language (HDL) like VHDL or Verilog.
2. **Granular Architectural Scaling:** Designers can implement the precise combination of modules required for a target workload. In resource-constrained applications, such as bare-metal microcontrollers or physical security coprocessors, omitting unnecessary extensions (e.g., floating-point or vector units) reduces total silicon area, power distribution requirements, and static leakage currents on FPGAs or ASICs.
3. **Toolchain Continuity:** Because the base ISA and standard extensions are immutable once ratified, software binaries remain functional across different generations of compliant hardware. Open-source compiler suites, including the GNU Compiler Collection (GCC) and LLVM, provide standard maintenance for RISC-V, removing dependencies on proprietary development software.
4. **Hardware-Software Co-design:** Open specifications grant complete visibility into the pipeline interface. This facilitates the implementation of custom hardware-enforced protection mechanisms—such as physical memory protection (PMP) policies or control-flow integrity verification units—directly into the processor’s state-tracking logic.

3.2 RISC-V Execution Model

The implementation of a RISC-V compliant processor requires the precise instantiation of the architectural state, which comprises the programmer-visible registers and the memory execution model. This state defines how data is stored, manipulated, and accessed across the hardware-software boundary. Furthermore, the architecture dictates deterministic mechanisms for handling anomalies and asynchronous events through a structured trap and exception framework, ensuring robust execution containment under bare-metal and privileged environments.

3.2.1 Register File Organization

For the 32-bit base integer architecture (RV32I), the fundamental execution state is maintained within a register file containing 32 general-purpose registers, designated x_0 through x_{31} , where the architectural register width (XLEN) is exactly 32 bits.

The register file exhibits the following structural constraints and characteristics:

- **Hardwired Zero Register (x_0):** The register x_0 is permanently grounded in hardware. Any read operation targeted at x_0 unconditionally returns a value of zero ($0x00000000$). Any write operation targeted at x_0 is discarded by the pipeline control logic, discarding the write-back data without altering the register state. This hardware design simplifies common operations, such as clearing registers, executing unconditional jumps, or implementing pseudo-instructions (e.g., matching a move instruction via an addition with x_0).
- **General-Purpose Registers (x_1 – x_{31}):** Registers x_1 through x_{31} are fully symmetric, read-write storage elements implemented via a multi-port static RAM block or flip-flop arrays within the processor core. While the standard software application binary interface (ABI) assigns specific roles to these registers (such as x_1 acting as the return address register ra , and x_2 acting as the stack pointer sp), the underlying hardware pipeline treats them identically during instruction decoding and execution phases.
- **Program Counter (PC):** The Program Counter is a distinct 32-bit architectural register that holds the byte address of the instruction currently being fetched or executed. The PC is not part of the general-purpose register file array and cannot be directly read or written via standard arithmetic instructions; instead, its state is altered implicitly via sequential execution increments (+4 bytes for standard 32-bit instructions) or explicitly via conditional branches and unconditional jump instructions.

3.2.2 Memory Model and Addressing

The RISC-V architecture specifies a linear, byte-addressable memory address space containing 2^{32} individual bytes, resulting in a total addressable range of 4 Gigabytes (GB).

The memory subsystem is governed by the following architectural specifications:

- **Data Granularity:** Memory operations can access data at granularities of a byte (8 bits), a half-word (16 bits), or a word (32-bit XLEN). Read operations targeting granularities smaller than a word can be signed (executing sign-extension across the remaining upper bits of the destination register) or unsigned (executing zero-extension).
- **Endianness:** By default, the base RISC-V specification adheres to a little-endian memory format. The least-significant byte of a multi-byte element is stored at the lowest memory address, and a word load or store maps the byte at address A to bits 7–0, address

$A + 1$ to bits 15–8, address $A + 2$ to bits 23–16, and address $A + 3$ to bits 31–24 of the target register.

- **Alignment Constraints:** RISC-V allows for either strict alignment or misaligned memory access support. In resource-constrained hardware implementations or pipelines targeting deterministic execution, misaligned accesses (e.g., a 32-bit word store at an address not divisible by 4) are typically configured to immediately raise an alignment exception. This shifts the handling responsibility to software trap handlers or enforces hardware restrictions to prevent multi-cycle memory controller penalties.

3.2.3 Control Flow, Exceptions, and Traps

Operational robustness and privilege separation within a RISC-V core depend on the trap handling mechanism. Control flow deviations are categorized into two distinct classes of events: synchronous exceptions and asynchronous interrupts.

1. **Synchronous Exceptions:** These are exceptional conditions directly attributable to the execution of an instruction within the current pipeline thread. Examples include:
 - Instruction address misaligned or access faults during the fetch phase.
 - Illegal instruction faults triggered by invalid opcodes or unsupported extension attempts during the decode phase.
 - Breakpoint exceptions triggered by debugging instructions.
 - Load/Store address misaligned or fault conditions occurring during the memory execution phase.
2. **Asynchronous Interrupts:** These are external, timing-independent events routed into the processor core from peripheral modules, such as a Universal Asynchronous Receiver-Transmitter (UART), hardware timers, or external inter-processor communication lines.

When either an exception or an interrupt condition is validated by the pipeline control logic, the processor executes an atomic state transition known as a *trap*. The execution pipeline is immediately flushed to isolate the offending or interrupted instruction, and control flow is redirected to a pre-configured trap vector address.

To facilitate hardware trap containment, the processor relies on architectural registers governed by the privilege specification, specifically interacting with Control and Status Registers (CSRs):

- **Machine Exception Program Counter (mepc):** A 32-bit register that automatically captures the address of the instruction that intercepted the exception, or the address of the next instruction to be executed when an interrupt occurs, serving as the return target.

- **Machine Cause Register (mcause):** A register containing an interrupt bit and an exception code field, detailing the exact structural reason for the trap invocation (e.g., value $0x2$ indicates an illegal instruction exception).
- **Machine Trap Value Register (mtval):** An auxiliary register that stores exception-specific information, such as the faulting memory address for a misaligned load/store or the raw instruction bit pattern for an illegal opcode trap.
- **Machine Trap-Vector Base-Address Register (mtvec):** A register that holds the base memory address of the software trap handler, supporting either a single direct address for all traps or a vectored array configuration where each interrupt code maps to a unique offset pointer.

The trap sequence terminates when the software handler executes a Machine Exception Return instruction (`mret`), which restores the execution state, reinstates the previous privilege level, and copies the contents of `mepc` back into the Program Counter.

3.3 RISC-V Encodings

The RISC-V base ISA enforces a fixed-length 32-bit instruction alignment, where every valid instruction must be situated at a memory boundary divisible by four bytes. The structure of the RISC-V encoding scheme is heavily optimized to minimize hardware decoding complexity in the processor core's Instruction Decode (ID) stage.

3.3.1 Base Instruction Formats

To accelerate register file access and pipeline scheduling, the architectural fields for the destination register (`rd`) and source registers (`rs1`, `rs2`) are held at constant bit positions across all instruction formats. This consistency permits the pipeline to initiate register file read cycles concurrently with the decoding of the operational codes (opcodes).

The base architecture categorizes all 32-bit instructions into six standard formats, defined as follows:

1. **R-Type (Register):** Utilized for purely register-to-register arithmetic, logical, and shift operations.
2. **I-Type (Immediate):** Utilized for register-immediate arithmetic, load operations, environment calls, and CSR instructions.
3. **S-Type (Store):** Utilized specifically for memory write operations, splitting the immediate field to preserve source register tracking fields.
4. **B-Type (Branch):** A specialized variation of the S-type format utilized for conditional branch operations.

5. **U-Type (Upper Immediate):** Utilized for wide 20-bit immediates allocated to upper-register loads (e.g., lui, auipc).
6. **J-Type (Jump):** A specialized variation of the U-type format engineered to maximize the offset range for unconditional branch instructions (e.g., jal).

The precise bit allocation for each of these six standard instruction formats is presented in Table 3.1.

Table 1: RISC-V Base Instruction Encoding Formats

Format	Bits [31:25]	Bits [24:20]	Bits [19:15]	Bits [14:12]	Bits [11:7]	Bits [6:0]
R-Type	funct7	rs2	rs1	funct3	rd	opcode
I-Type	imm[11:0]		rs1	funct3	rd	opcode
S-Type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B-Type	imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode
U-Type	imm[31:12]				rd	opcode
J-Type	imm[20 10:1 11 19:12]				rd	opcode

3.3.2 Field Descriptions

Every structural block inside the 32-bit instruction boundary serves a specific decoding function for the control logic:

- **opcode [6:0]:** A 7-bit primary identifier located at the lowest bits of every instruction. It determines the base instruction format and execution category. For standard 32-bit base instructions, the lowest two bits (opcode[1:0]) are hardwired to 11.
- **rd [11:7]:** A 5-bit destination register identifier selecting registers x_0 to x_{31} .
- **funct3 [14:12]:** A 3-bit auxiliary opcode space used in conjunction with the primary opcode to distinguish variations of a given instruction category (e.g., separating an addition ADD from an exclusive-OR XOR).
- **rs1 [19:15]:** A 5-bit primary source register identifier.
- **rs2 [24:20]:** A 5-bit secondary source register identifier.
- **funct7 [31:25]:** A 7-bit high-order functional field utilized primarily in R-type structures to specify secondary mathematical transformations (e.g., distinguishing an arithmetic right shift SRA from a logical right shift SRL).

3.3.3 Immediate Encoding

To reduce hardware routing congestion and timing delays inside the execution pipeline, all RISC-V immediate values are structured to keep the sign bit aligned at bit 31 of the total instruction word. As a result, the sign-extension circuitry can tap directly into instruction bit 31, allowing sign-propagation to execute in parallel with decoding.

For formats that utilize scrambled or non-contiguous immediate variants (B-Type and J-Type), the bits are re-ordered within the instruction word to maximize commonalities with standard S-type and U-type layouts. Furthermore, all RISC-V immediates are sign-extended to a full 32-bit register width before being presented to the Arithmetic Logic Unit (ALU) or the execution pipeline.

The hardware implementation maps the incoming raw instruction bits ($\text{inst}[31:0]$) to their corresponding positions in the final 32-bit sign-extended literal (Imm_{32}). This structural reconstruction is governed deterministically by the format rules outlined in Table 3.2.

Table 2: RISC-V Immediate Generation Mapping

Format	Bits [31:20]	Bits [19:12]	Bit 11	Bits [10:5]	Bits [4:1]	Bit 0
I-Type	$\text{inst}[31]$ (sign-extended)		$\text{inst}[31]$	$\text{inst}[30:25]$	$\text{inst}[24:21]$	$\text{inst}[20]$
S-Type	$\text{inst}[31]$ (sign-extended)		$\text{inst}[31]$	$\text{inst}[30:25]$	$\text{inst}[11:8]$	$\text{inst}[7]$
B-Type	$\text{inst}[31]$ (sign-extended)		$\text{inst}[7]$	$\text{inst}[30:25]$	$\text{inst}[11:8]$	0
U-Type	$\text{inst}[31:20]$	$\text{inst}[19:12]$	0	0	0	0
J-Type	$\text{inst}[31]$ (sign-ext)	$\text{inst}[19:12]$	$\text{inst}[20]$	$\text{inst}[30:25]$	$\text{inst}[24:21]$	0

3.4 Base Instruction Set

The core functionality of the SoC execution engine is governed by the RV32I base integer instruction set. The designation RV32I specifies a standard 32-bit address and data word width ($\text{XLEN} = 32$) operating over an immutable integer base computing profile. The instruction set represents the absolute minimum algorithmic primitives required to construct a valid compiler target capable of executing general-purpose computation, structural control flow, and memory access patterns.

3.4.1 Architectural Scope and Design Motivations

The RV32I variant is engineered to maintain microarchitectural simplicity within the processor core. By removing advanced mathematical primitives—such as hardware floating-point hardware extensions (F/D) or complex vector logic—the execution hardware reduces decode stage propagation delays and keeps total gate-count low. This minimal baseline profile establishes a deterministic execution context, allowing the system to host custom control-flow security

enhancements (e.g., Zicfilp and Smcfiss) without experiencing decode logic path scaling bottlenecks.

The baseline execution profile divides the computational workload into distinct functional groups:

- **Computational Operations:** These perform register-to-register or register-to-immediate transformations via the Arithmetic Logic Unit (ALU). They comprise logical operations (AND, OR, XOR), basic arithmetic (ADD, SUB), and logical/arithmetic bit shifts (SLL, SRL, SRA).
- **Load and Store Operations:** These manage data movement between the core register file and the memory controller. Loads (LB, LH, LW, LBU, LHU) retrieve variables from memory, while stores (SB, SH, SW) commit data to the unified memory array.
- **Control Flow Operations:** Unconditional jumps (JAL, JALR) and conditional branches (BEQ, BNE, BLT, BGE, BLTU, BGEU) alter the program counter based on comparative calculations, forming the basis for loops, function invocation, and branch execution.
- **System Operations:** System instructions (ECALL, EBREAK, FENCE) interface directly with the privilege architecture or execute ordering constraints on memory operations across system interfaces.

3.4.2 RV32I Instruction Set Bit-Level Specifications

Table 3.3 presents the structural field mapping and hardware opcode patterns for the complete set of base integer instructions implemented within the processor pipeline.

Table 3: RV32I Base Instruction Set Bit Allocations

Bits [31:25]	Bits [24:20]	Bits [19:15]	Bits [14:12]	Bits [11:7]	Bits [6:0]	Mnemonic
imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
imm[20 10:1 11 19:12]				rd	1101111	JAL
imm[11:0]		rs1	000	rd	1100111	JALR
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE

Continued on next page

Table 3 – continued from previous page

Bits [31:25]	Bits [24:20]	Bits [19:15]	Bits [14:12]	Bits [11:7]	Bits [6:0]	Mnemonic
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL

Continued on next page

Table 3 – continued from previous page

Bits [31:25]		Bits [24:20]	Bits [19:15]	Bits [14:12]	Bits [11:7]	Bits [6:0]	Mnemonic
0100000		rs2	rs1	101	rd	0110011	SRA
0000000		rs2	rs1	110	rd	0110011	OR
0000000		rs2	rs1	111	rd	0110011	AND
fm	pred	succ	rs1	000	rd	0001111	FENCE
1000	0011	0011	00000	000	00000	0001111	FENCE.TSO
0000	0001	0000	00000	000	00000	0001111	PAUSE
000000000000			00000	000	00000	1110011	ECALL
000000000001			00000	000	00000	1110011	EBREAK

3.5 RISC-V Ratified Extensions

To facilitate hardware state monitoring, exception containment, and advanced security execution policies, the baseline integer computing model is augmented with ratified architectural extensions. The integration of these modular extensions directly alters the control logic and state validation operations within the core execution pipeline.

3.5.1 Zicsr Extension

The Zicsr extension defines the standard structural interface for accessing the Control and Status Registers (CSRs), which govern the processor's administrative status, exception response, and privileged runtime modes. CSR operations are atomic read-modify-write instructions that bypass the general-purpose register file to communicate directly with internal hardware state registers.

Table 3.4 outlines the bit-level fields and primary opcodes for the complete set of standard CSR access instructions.

Table 4: Zicsr Instruction Set Bit Allocations

Bits	Bits	Bits	Bits [11:7]	Bits [6:0]	Mnemonic
csr[11:0]	rs1	001	rd	1110011	CSRRW
csr[11:0]	rs1	010	rd	1110011	CSRRS
csr[11:0]	rs1	011	rd	1110011	CSRRC
csr[11:0]	uimm[4:0]	101	rd	1110011	CSRRWI
csr[11:0]	uimm[4:0]	110	rd	1110011	CSRRSI
csr[11:0]	uimm[4:0]	111	rd	1110011	CSRRCI

The microarchitectural core instantiates a dedicated architectural register matrix to trace and main-

tain execution conditions, interrupt masks, and security barriers. The implemented registers comprise:

- **Machine Vendor ID Register (mvendorid):** A 32-bit read-only hardware register initialized to a constant zero value (0x00000000), designating a non-commercial, open-source microarchitectural implementation.
- **Machine ISA Register (misa):** A 32-bit read-only state register fixed at value 0x40000100. The upper two bits (bits[31:30] = 01) specify an active 32-bit architectural width (XLEN=32), while bit 8 is set to high, explicitly declaring the base RV32I instruction set profile.
- **Machine Status Bits (mstatus):** Distributed control flags tracking active and previous execution permissions:
 - MIE (mstatus[3]): Machine Interrupt Enable bit governing global hardware interrupt validation.
 - MPIE (mstatus[7]): Machine Previous Interrupt Enable bit, preserving the pre-trap state of the MIE flag during context switches.
- **Machine Trap Vector Register (mtvec):** Holds the base address pointer and alignment modes for routing control flow upon exception or external interrupt verification.
- **Machine Trap Value Register (mtval):** Captures contextual trap information, such as illegal instruction bit patterns or faulty data references.
- **Machine Exception Program Counter (mepc):** A dedicated tracking pointer that isolates the address of the instruction that encountered the fault condition or interrupt transition.
- **Machine Cause Register (mcause):** Details the technical reason for the trap state, distinguishing synchronous computational exceptions from asynchronous peripheral interrupts via an active high-order bit.
- **Machine Interrupt Enable Register (mie):** A 32-bit register containing individual mask bits to enable or disable target interrupt categories (e.g., timer, software, or external lines).
- **Machine Cycle Counter (mcycle):** A 32-bit hardware performance counter that increments unconditionally on every active clock cycle.
- **Machine Instructions Retired Counter (minstret):** Tracks system performance metrics by incrementing exclusively when an instruction successfully completes all pipeline stages and commits its results.
- **Machine Scratch Register (mscratch):** A dedicated storage register utilized by software trap handlers to temporarily preserve execution context without altering active general registers.
- **Machine Security Configuration Register (mseccfg):** Tracks advanced architectural security flags, governing physical memory protection rules and security configuration locks.

- **Physical Memory Protection Configuration (pmpcf_{g0}):** Establishes access control attributes (Read, Write, Execute, and Address Matching settings) for the hardware memory protection blocks.
- **Physical Memory Protection Address Registers (pmpaddr₀ to pmpaddr₃):** Store the physical base and bound address ranges utilized by the filtering logic to validate memory access requests.
- **Indirect CSR Interface Registers (miselect, mireg, mireg₂):** Provide a structured indirect selector and data register mechanism to read and alter extended microarchitectural state vectors without allocating excessive primary CSR address spaces.
- **PMP Execution/Entry Flags (pmp_e_bits):** An internal hardware state register that mirrors active PMP execution permissions to ensure single-cycle check operations within the pipeline memory stages.
- **Shadow Stack Pointer (ssp):** A dedicated register that holds the current top-of-stack memory reference for the hardware-enforced shadow stack sub-system, isolating return address contexts from the standard stack pointer framework.

3.5.2 Zicfilp Extension

The Zicfilp extension establishes hardware-enforced forward-edge Control-Flow Integrity (CFI). Its primary objective is to eliminate arbitrary indirect jump-oriented programming (JOP) exploits by ensuring that indirect control flow transfers (executed via JALR instructions) target only valid, pre-defined entry points within the application binary.

Expected Landing Pad (ELP) State The core pipeline incorporates an internal 1-bit state element called the Expected Landing Pad (ELP) state machine. This state machine tracks whether the processor is executing a standard sequential stream or validating a dynamic control flow transition. The tracking states are defined as:

- NO_LP_EXPECTED (0): The default execution state. The core interprets incoming operations normally.
- LP_EXPECTED (1): The validation state triggered immediately after an indirect jump or call is evaluated. In this state, the next sequential instruction presented to the decode stage must be a valid landing pad primitive.

Landing Pads and the lpad Instruction A landing pad is a code entry point marked by a specialized landing pad instruction (lpad). When the ELP state transitions to LP_EXPECTED, the hardware monitors the incoming instruction word. If the instruction following the indirect jump is not a valid lpad instruction, the control logic halts execution and raises a software-check exception (mcause = 18) with a trap value (mtval) of 2.

To support advanced structural checks, the landing pad mechanism implements label verification. A 20-bit function signature or hash is pre-loaded into register x7 at the calling site. The lpad instruction

embeds a matching 20-bit literal known as the Landing Pad Label (LPL). If the immediate field embedded within the instruction is non-zero, the processor automatically executes a hardware comparison against register x7. A mismatch immediately generates a software-check trap.

Instruction Encoding To maintain backward compatibility with legacy processors that lack CFI hardware support, the lpad instruction is encoded within the existing code point allocated to the AUIPC major opcode, specifically targeting a destination register identifier of x0 ($rd = 00000$). On hardware platforms without the Zicfilp extension, this configuration executes benignly as a standard architectural hint no-op (HINT NOP), whereas the tracking core parses it as a mandatory control enforcement boundary.

The bit-level structural layout of the landing pad instruction is detailed in Table 3.5.

Table 5: Zicfilp Landing Pad Instruction Encoding

Bits [31:12]	Bits [11:7]	Bits [6:0]	Mnemonic
Landing Pad Label	00000 ($rd = x0$)	0010111 (AUIPC)	lpad

3.6 RISC-V Draft Extensions

To support scalable physical memory protection and expand backward-edge control-flow integrity across privileged states, the core execution framework is augmented with modular draft extensions. These specifications expand the indirect register aliasing space and introduce rigid hardware tracking mechanisms for higher privilege tiers.

3.6.1 Smpmpind Extension

The Smpmpind extension defines a structured architectural interface that facilitates indirect access to Physical Memory Protection (PMP) registers. It depends completely on the Smcsrind (CSR Indirect Access) extension, utilizing its core indirect selector configuration to address PMP registers without exhausting the primary CSR address space.

During execution, the target PMP entry index is pre-loaded into the indirect selector register (miselect). The data registers mireg and mireg2 are then utilized to alter or inspect the underlying tracking states. The specific microarchitectural register routing is outlined in Table 3.6.

Table 6: Smpmpind Indirect PMP Register Target Allocations

Indirect Selector State	Indirect Data Register	Mapped PMP Register Block
miselect [Target Entry]	mireg	pmpaddr Instruction Target
miselect [Target Entry]	mireg2	pmpcfg Control Interface
miselect [Target Entry]	mireg3 to mireg6	Reserved for Future Standards

The newly defined pmpcfg register scales across the full architectural width (MXLEN). The lowest 8 bits of this register serve as a direct functional alias of the standard 8-bit PMP configuration field

accessed via conventional direct CSR instructions. The microarchitectural locking state semantics (L bit) remain entirely unaffected by this indirect mechanism.

Extended structural capabilities are introduced via the high-order bits of the configuration layout:

- **Extended-Attributes (E) Field (bit [MXLEN-1]):** Governs the active interpretation protocol of the primary access control bits.
 - When $E = 0$, the Execute, Write, and Read (XWR) access control flags are interpreted normally, maintaining strict backward compatibility with the baseline architecture.
 - When $E = 1$, all baseline encodings of the XWR bit fields are decoupled and treated as reserved unless explicitly specified by another concurrent architectural extension. Attempting to access a PMP region while $E = 1$ under a reserved XWR configuration immediately generates an access fault exception corresponding to the initiating memory reference type (Load, Store/AMO, or Instruction Fetch).

3.6.2 Smcfiss Extension

The Smcfiss extension introduces hardware-enforced backward-edge Control-Flow Integrity (CFI) for Machine-mode (M-mode) software execution. By instantiating a separate, hardware-protected shadow stack structure, the pipeline eliminates return-oriented programming (ROP) exploits within the highest execution privilege tier. The Smcfiss extension maintains explicit architectural dependencies on both the Zicfiss and Smpmpind extensions.

Activation and Lock Interlocking M-mode shadow stack functionality is governed by a dedicated configuration field, MSSE (Machine Shadow Stack Enable), instantiated within the Machine Security Configuration Register (mseccfg). The internal shadow-stack-enabled (xSSE) architectural tracking state directly reflects the value of mseccfg.MSSE. On platforms where Smcfiss is omitted, the MSSE bit field is hardwired to a read-only zero value.

To guarantee execution security when the Smepmp extension is present, an administrative interlock is enforced: if the configuration write lock is active ($\text{pmpcfg.L} = 1$) on any PMP entry while the rule-locking bypass is disabled ($\text{mseccfg.RLB} = 0$), any subsequent software writes to the mseccfg.MSSE field are suppressed by hardware logic until a full system PMP reset occurs.

Memory Region Definition When mseccfg.MSSE is asserted to 1, a PMP entry configured with an active extended attribute bit ($E = 1$) and an access pattern encoding of $\text{XWR} = 010$ is uniquely interpreted by the memory management unit as a dedicated M-mode shadow stack region. If mseccfg.MSSE is cleared to 0 while $E = 1$, the $\text{XWR} = 010$ encoding reverts to a reserved state.

Shadow Stack Primitives (Zicfiss Integration) The Smcfiss extension leverages the core architectural primitives defined by the Zicfiss extension to manipulate the M-mode shadow stack structure. These instructions bypass normal data-write pathways to interface directly with the memory region marked by the $\text{XWR} = 010$ configuration. They allow the pipeline to push, pop, and audit function return addresses atomically.

Table 3.7 details the hardware-enforced behavior of the primary `Zicfiss` instructions utilized within the `Smcfiss` environment.

Table 7: `Zicfiss` Shadow Stack Instruction Primitive Summary

Mnemonic	Operands	Functional Microarchitectural Operation
<code>sspush</code>	<code>rs2</code>	Decrements the shadow stack pointer (<code>ssp</code>) and pushes the register value.
<code>sspopchk</code>	<code>rs1</code>	Pops the top value from <code>ssp</code> , compares it to <code>rs1</code> , and increments <code>ssp</code> .
<code>ssrdp</code>	<code>rd</code>	Reads the current absolute memory address matching <code>ssp</code> into <code>rd</code> .
<code>ssmoswap</code>	<code>rd</code> , <code>rs2</code>	Swaps the active machine-mode <code>ssp</code> pointer state with a general register.

Privileged Access Rules Strict, hardware-enforced isolation rules govern all memory references targeting an active M-mode shadow stack memory region based on the effective privilege level:

1. **Effective Privilege Mode is M:**

- **Shadow Stack Instructions:** Permitted to access only explicitly designated M-mode shadow stack regions. An attempt by a shadow stack instruction (such as `sspush` or `sspopchk`) to access any other memory space generates a store/AMO access fault exception.
- **Standard Non-Shadow Stack Instructions:** Expressly prohibited from performing write operations to an M-mode shadow stack area. Any standard store or atomic memory operation (AMO) targeting this region raises a store/AMO access fault exception.
- **Read Verification:** The M-mode shadow stack region remains readable by both explicit hardware load instructions and native shadow stack primitives.
- **Implicit Control Accesses:** Any implicit memory reference, including standard instruction fetches targeting the shadow stack boundary, is blocked and triggers an access fault exception matching the faulting access type.

2. **Effective Privilege Mode is Less than M ($\neq$ M):**

- Any memory access request (Load, Store, or Fetch) originating from a lower privilege mode targeting an M-mode shadow stack region is blocked at the pipeline memory stage, generating an immediate access fault exception corresponding to the initiation type.

CHAPTER 4

HARDWARE DESIGN

4.1 System-on-Chip (SoC) Structure

The eSC-V is an extensively verified, hardware-enforced Control-Flow Integrity (CFI) System-on-Chip (SoC) designed entirely in VHDL. Conceived for bare-metal, Machine-mode (M-mode) microcontroller operations, the architecture fundamentally shields software against advanced exploitation techniques such as Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP).

At the apex of its structural hierarchy, the SoC acts as the integrating wrapper (defined within `soc.vhd`) that tightly synchronizes the 5-stage pipelined RV32I processor core with essential internal peripherals, unified memory, and memory-mapped address buses.

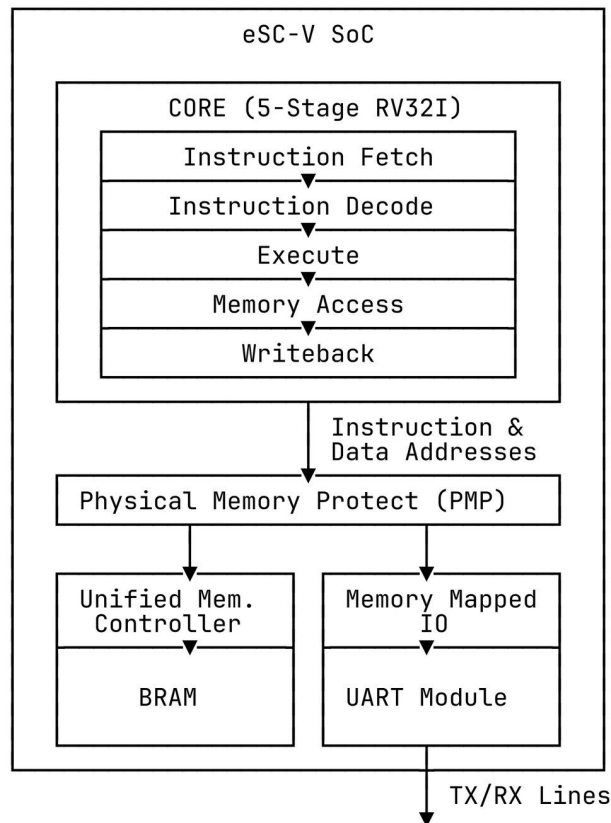


Figure 1: eSC-V SoC Block Diagram

4.1.1 Core Philosophy and Hardware Ecosystem

The physical synthesis target for the eSC-V SoC is the Tang Primer 20K FPGA. The overarching development ecosystem leans heavily on open-source implementations to ensure transparency and reproducibility. While the top-level VHDL generic configuration parameters specify a default system clock (`G_CLK_FREQ`) of 27 MHz and a UART baud rate (`G_BAUDRATE`) of 921,600 bps, rigorous synthesis optimizations permit the processor to run flawlessly at an accelerated 54 MHz.

The build and deployment process utilizes an entirely open-source FPGA toolchain, invoking Yosys to synthesize the VHDL elements and nextpnr-himbaechel for place-and-route operations onto the Tang Primer 20K architecture. The SoC build system intrinsically depends on the Nix package manager to or-

chestrate a reproducible development environment, which manages the custom GCC compiler required to parse the newly integrated CFI instruction extensions (such as `Zicfilp`, `Smcfiss`, and `Smpmpind`).

4.1.2 Power-On Reset (POR) Circuitry

Physical hardware constraints inherently shape SoC design decisions. The targeted Tang Primer 20K FPGA board presents a distinct structural limitation: it lacks easily accessible external Input/Output (I/O) pins that can be comfortably utilized for a standard hardware reset.

To counteract this, the eSC-V SoC explicitly incorporates a dedicated internal Power-On-Reset (POR) generation circuit. This specialized, internally contained circuit takes immediate responsibility for holding the system in reset upon initial power delivery. It ensures that all highly sensitive synchronous logic—including pipeline registers, instruction state machines, and UART transceivers—initializes into a deterministic, safe state the moment the FPGA bitstream is actively loaded.

4.1.3 Address Space Partitioning and Memory Management

To orchestrate data flow between the central processor and its subsidiary components, the SoC enforces a rigid, predefined memory map, dividing the physical address space into two strictly separated regions:

Main Memory Space (Instruction and Data): Utilizing a generic parameter (`G_RAM_SIZE`), the SoC instantiates a default capacity of 8,192 32-bit words, culminating in 32 KB of total Block RAM (BRAM) allocation. This memory maps strictly to the base address `0x80000000` and concludes precisely at $0x80000000 + (G_RAM_SIZE \times 4) - 1$.

Despite utilizing a single, unified BRAM array at the physical level, the SoC interfaces with it using Harvard architecture abstraction. This permits the processor's Instruction Fetch (IF) and Memory (MEM) stages to execute simultaneous, dual-port reads and writes. Prior to synthesis or simulation, the system application—whether that is a C-based `libc` test, a vulnerable CFI target, or ASCII Tetris—is pre-loaded directly into this block via a predefined hex initialization file (`code.hex`).

Memory-Mapped I/O Space (MMIO): The external hardware peripherals are controlled entirely through absolute address mapping, rather than utilizing specialized CPU I/O instructions. This hardware interface is positioned directly at the base of the address spectrum, starting at `0x00000000`.

4.1.4 Synchronous Address Decoding and Latency Matching

A critical synchronization challenge within the SoC's structural logic is bridging the processor core with memory arrays that exhibit strict hardware delays. When synthesizing to FPGA Block RAM (BRAM), the memory introduces an unyielding 1-clock-cycle synchronous read latency.

To prevent catastrophic data misalignment, pipeline stalls, or bubbles in the Writeback (WB) stage, the SoC must perfectly mirror this latency across all of its routing multiplexers. The SoC achieves this via synchronous address decoding. When the core's MEM stage requests a read operation, the SoC evaluates the requested address and utilizes a dedicated hardware flip-flop, registered as `r_access_was_io`,

to memorize the exact physical destination (whether it was routed to the BRAM array or to the MMIO peripheral).

Exactly one clock cycle later, as the BRAM makes its internal data available, this `r_access_was_io` flag drives a return multiplexer. This ensures that the incoming data stream to the CPU's register file smoothly and accurately toggles between BRAM data and peripheral data without requiring complex stalling logic.

4.1.5 The Universal Asynchronous Receiver-Transmitter (UART) Peripheral

As the primary external physical interface for the SoC, a custom UART transceiver is instantiated and tightly integrated directly into the `soc.vhd` top-level architecture. This peripheral acts as the single umbilical cord for the system, facilitating serial debugging, exploit payload delivery for CFI testing, and user keyboard inputs for compiled applications.

The SoC maps this transceiver across two 32-bit registers, ensuring precise, granular control over serial traffic:

- **UART Data Register (0x00000000):** This is the bidirectional data conduit. If the CPU initiates a memory store (write) to this address, the SoC intercepts the byte, queues it into the UART module, and immediately pushes it across the physical TX line (assuming the module is idle). If the CPU performs a memory load (read), the SoC fetches the most recently latched byte captured from the physical RX line.
- **UART Status Register (0x00000004):** Because serial communication operates asynchronously and at significantly slower speeds than the 54 MHz core, the processor must monitor the UART's internal state machine via this register.
 - **Bit 0 (TX Ready):** This bit serves as a hardware lock. A logical 1 indicates the transmission hardware has finished shifting out previous data and can safely accept a new byte. A 0 indicates the line is actively busy, and software must wait.
 - **Bit 1 (RX Valid):** This bit acts as a data-available flag. A logical 1 alerts the CPU that a new, valid byte has arrived. To ensure system stability and prevent the CPU from rapidly polling and consuming the same character multiple times, the SoC enforces a strict acknowledgment mechanism: the software must manually execute a write instruction to this Status Register to forcibly clear the RX Valid flag back to 0.

4.1.6 Debug Hooks and Compliance Verification

The top-level structure of the SoC goes beyond simply connecting logic gates for software execution; it is specifically modeled to support continuous formal verification. The SoC incorporates dedicated architectural tap points designed explicitly for the RISC-V Compatibility Framework (RISCOF).

To facilitate this, the `soc.vhd` module intentionally intercepts and exposes internal, real-time data memory signals outward to the top-level pins. External testbenches (`tb_soc_risconf`) latch onto these

signals, allowing software environments to continuously monitor memory state mutations cycle-by-cycle. This intricate transparency guarantees that the SoC's hardware behavior rigorously aligns with the official Sail and Spike formal models established by RISC-V International.

4.2 CPU Core

Acting as the central coordinator of the eSC-V processing environment, the CPU core is structurally defined within `core.vhd`. The core implements a single-issue, in-order, 5-stage pipeline executing the RV32I base integer instruction set and the Zicsr system register extension. The execution pipeline is divided into five distinct stages: Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Writeback (WB). These stages are isolated from one another by four synchronous pipeline registers—specifically the IF/ID, ID/EX, EX/MEM, and MEM/WB registers—which latch state transitions on every rising clock edge. Under normal conditions, instructions flow sequentially through this pipeline, taking one clock cycle per stage to achieve a throughput of one instruction per cycle. Feedback paths are integrated to close the control and data loops, including data writeback paths routing register write data from the Writeback stage back to the Register File in the Decode stage, and target redirection signals routing computed execution targets from the Execute stage back to the PC register in the Fetch stage.

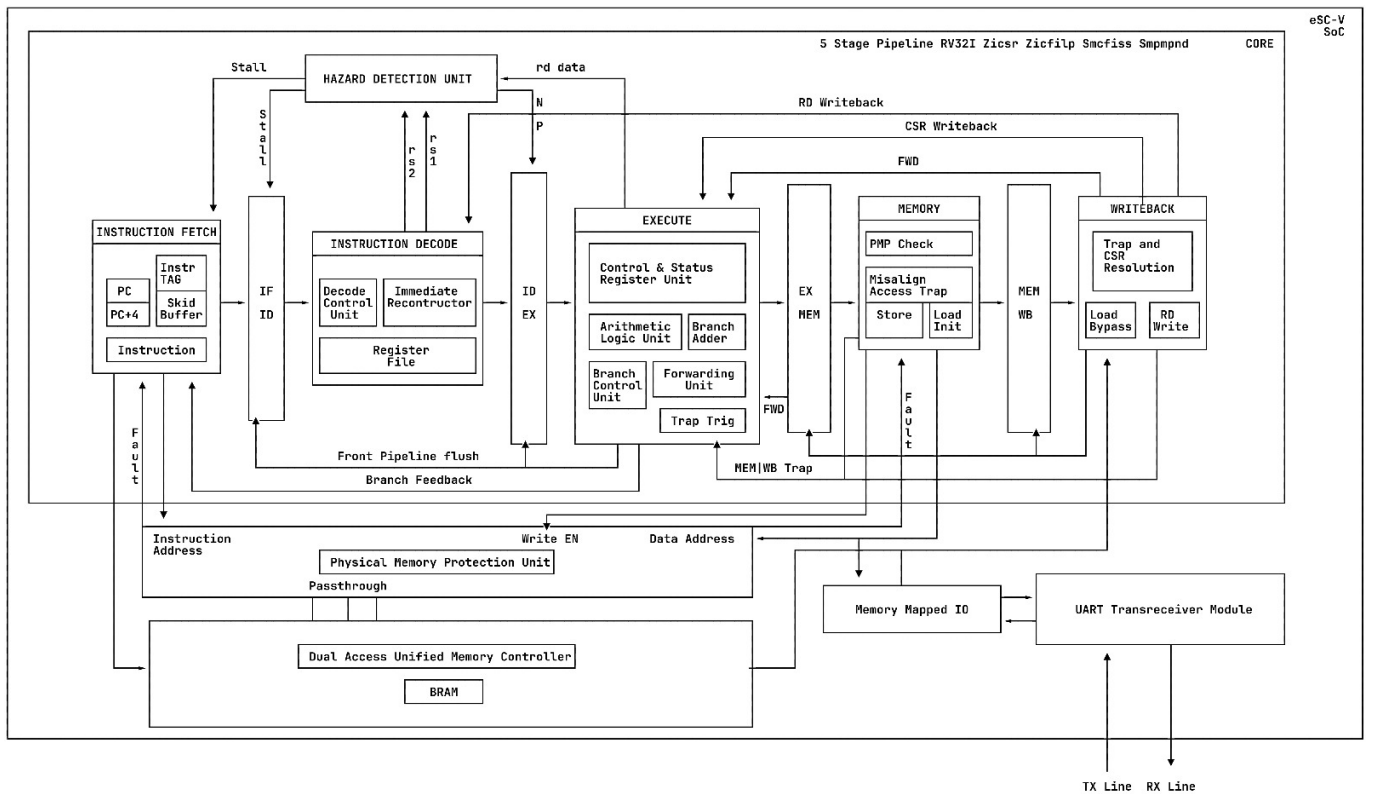


Figure 2: eSC-V detailed Block Diagram

To secure this pipeline against control-flow hijack attacks and privilege escalation, the core integrates security engines directly into the hardware stages. Memory protection is managed globally by an out-of-pipeline Physical Memory Protection (PMP) unit (`pmp_unit.vhd`). The PMP unit is instantiated outside the pipeline register chain to perform combinational address filtering on both memory interfaces using

a unified configuration register set. For instruction fetches, the PMP unit checks the fetch address and asserts a fetch fault if a violation is detected. For data accesses, it checks data read/write addresses and asserts data read or write access faults. These faults are aligned with their respective instructions using delay registers and are propagated down the pipeline, ensuring precise exception handling. The PMP unit also identifies shadow stack memory accesses, restricting access to hardware-managed pointer updates.

Data hazards between dependent instructions are resolved dynamically without pipeline stalls using the Forwarding Unit in the Execute stage. However, when an instruction in the Decode stage requires data from a load instruction currently in the Execute stage, a load-use hazard occurs. Because the load data is not retrieved from memory until the Memory Access stage, forwarding alone cannot resolve the hazard. To handle this, the core implements a Hazard Detection Unit (`hazard_detection_unit.vhd`). When a load-use hazard is detected, the unit asserts a pipeline stall signal, which freezes the PC register in the Fetch stage and blocks updates to the IF/ID pipeline register. Simultaneously, the unit flushes the ID/EX pipeline register to insert a NOP bubble into the Execute stage. This stalls the dependent instruction by one clock cycle, allowing the load instruction to complete memory access and forward the data.

Control hazards arising from branches, jumps, and traps are resolved using a two-stage flush mechanism. The flush lines are divided into a Front-Pipeline Flush and a Back-Pipeline Flush. The Front-Pipeline Flush is driven by the Execute stage and is asserted when a conditional branch is taken, a jump instruction is executed, or a critical CSR write is detected. When active, it resets the IF/ID and ID/EX pipeline registers to NOP instructions, discarding speculative instructions fetched from the wrong path before they can modify the processor state. The Back-Pipeline Flush is driven by the Writeback stage and is asserted when a precise exception commits (such as a shadow stack return address mismatch, a PMP access fault, a memory misalignment, or an environment call) or when a critical CSR write commits. When active, it resets the EX/MEM and MEM/WB pipeline registers to NOP instructions, while the Fetch stage redirects the PC to the trap vector address. Splitting the flush signals prevents out-of-order execution during exceptions, avoids state leakage, and ensures that traps or configuration updates cleanly sweep the pipeline from both ends, keeping the stages synchronized under changing security policies.

4.2.1 Instruction Fetch (IF) Stage

Serving as the architectural gateway and control flow entry point of the eSC-V processor, the Instruction Fetch (IF) stage is structurally encapsulated within the `instruction_fetch_stage.vhd` entity. While later stages in the pipeline are responsible for decoding, executing, and committing instructions, the IF stage is dedicated to managing the Program Counter (PC), calculating the next sequential instruction address, generating memory fetch requests, and aligning instruction metadata.

Because the core integrates advanced Control-Flow Integrity (CFI) protections, the IF stage must also manage the expected landing pad state and handle physical memory protection (PMP) violations. To handle synchronous memory latency and pipeline stalls without losing instruction data, the stage implements dedicated alignment pipelines and a hardware skid buffer.

4.2.1.1 Program Counter (PC) Update & Control Flow Redirection

The core of the IF stage's control path is the Program Counter update process. Governed by the global clock and synchronous reset, the PC register is initialized to the designated reset address constant, defined as `0x80000000` in the global package file `common.vhd`.

- **Sequential Fetching:** Under normal, non-stalled execution conditions, the PC register increments sequentially by four bytes on every rising clock edge.
- **Stall Conditions:** If the downstream pipeline asserts a stall signal (such as during a data hazard or memory wait state), the PC update process freezes, preserving the current address request.
- **Control Flow Redirections:** When a redirect is requested by the Execution stage (such as a taken branch, absolute jump, exception trap, or security redirect) via the execution-to-fetch bus (`common.vhd`), the PC update process overrides the sequential increment. It immediately loads the redirection address and updates the expected landing pad tracking register to the state requested by the Execution stage.

4.2.1.2 Memory Latency & State Alignment Logic

The eSC-V core is designed to interface with synchronous BRAM-based instruction memory, which introduces a one-cycle read latency. Consequently, the instruction data returned from memory arrives exactly one clock cycle after its corresponding address is requested.

To prevent data mismatch, the stage implements an alignment pipeline:

- **PC Alignment:** The PC of the instruction currently being requested is latched into an aligned PC register. This ensures that when the instruction data arrives one cycle later, it can be paired with its correct PC value before being passed to the Decode stage.
- **Security State Alignment:** The PMP access fault flag and the expected landing pad state are also routed through matching one-cycle delay registers. This aligns the security metadata with the returning instruction data.
- **Redirection Clears:** If a control redirection is initiated, the alignment logic clears the PMP fault register and resets the aligned PC to zero. This prevents stale fault states or invalid PCs from propagating to the Decode stage.

4.2.1.3 The Skid Buffer (Hazard Mitigation)

In a pipelined architecture, a stall in a downstream stage prevents the IF stage from pushing new instructions into the pipeline. However, due to the one-cycle synchronous memory latency, the instruction memory is already processing a fetch cycle and will output the next instruction on the subsequent clock edge. Because the downstream stages cannot accept new instructions, this incoming instruction would be lost when the memory output changes.

To resolve this hazard, the IF stage implements a hardware Skid Buffer:

- **Data Capture:** When a pipeline stall is active and the skid buffer is empty, the skid buffer captures the incoming instruction data, the aligned PC, the aligned PMP fault status, and the aligned expected landing pad state from the alignment registers. It then asserts a skid validation flag.
- **Data Bypassing:** When the pipeline stall is cleared, the skid buffer deasserts its validation flag. While the validation flag is active, the stage's output multiplexer bypasses the raw inputs from memory and instead routes the saved values from the skid buffer to the Decode stage.
- **Redirection Handling:** If a control flow redirection occurs, any data stored in the skid buffer is immediately invalidated, as the redirection flushes all speculative instructions in flight.

This implementation acts as a temporary, one-deep hardware queue that prevents instruction loss during asynchronous pipeline stalls.

4.2.1.4 Pipeline Flushes and Redirection Logic

Because branch and jump decisions are resolved in the Execution stage (two stages downstream from Fetch), a taken redirect means that the instructions currently in flight in the Fetch and Decode stages are speculative and invalid. The pipeline must flush these instructions to prevent them from modifying the processor state.

To handle flushes:

- **Flush Pending Register:** A dedicated register records when a redirection request is received from the Execution stage.
- **NOP Insertion:** On the clock cycle following a redirection, the output multiplexer overrides the instruction output, replacing it with a hardware No-Operation (NOP) instruction, defined as `0x00000013` (an integer add-immediate of register zero with itself).
- **Metadata Clearing:** Simultaneously, the expected landing pad flag is cleared to zero, and the program counter output is set to zero. This ensures that the speculative instruction is ignored by the Decode stage.

4.2.1.5 Output Multiplexer & PMP Fault Propagation

The final stage of the Instruction Fetch architecture is the output multiplexer, which packages the instruction and its associated flags into the pipeline bus (`common.vhd`) connected to the IF/ID pipeline register.

- **Multiplexer Logic:** The multiplexer selects between the skid buffer registers (if the skid validation flag is set) and the active alignment registers under normal execution.
- **PMP Fault Handling:** If the PMP unit detects an access violation for the current instruction address, the aligned PMP fault flag is asserted. When this flag is active, the output multiplexer overrides the fetched instruction, replacing it with a NOP. Simultaneously, it sets the instruction's fault tag to `IF_ACCESS_FAULT` (type `common.vhd`).

- **Safe Trap Propagation:** Rather than halting the processor immediately on a PMP violation, propagating the fault tag down the pipeline ensures that the access violation is trapped synchronously when the instruction reaches its execution phase, maintaining precise exception handling.
- **Normal Output:** If no flushes are pending and no PMP violations are detected, the multiplexer routes the fetched instruction along with its aligned PC, next-sequential PC, and landing pad expectation state to the Decode stage, marking the fault tag as VALID.

4.2.2 Instruction Decode (ID) Stage

Serving as the parser and logical router of the pipeline, the Instruction Decode (ID) stage is structurally encapsulated within the `instruction_decode_stage.vhd` wrapper entity. While the preceding Fetch stage is primarily responsible for retrieving raw instruction bits from memory, the Decode stage translates these bits into control vectors, routes source registers, and reconstructs immediate operands.

To manage this logical transition, the top-level wrapper coordinates three subsidiary hardware modules:

- The Instruction Decode Control Unit (`id_control_unit.vhd`)
- The Register File (`register_file.vhd`)
- The Immediate Reconstruction Unit (`immediate_reconstruct_unit.vhd`)

The ID stage wrapper manages the interfaces between these modules. It ingests instructions and PC values from the Fetch stage via the `common.vhd` bus, extracts instruction fields, manages register read/write paths, and propagates the decoded control signals and data to the Execute stage via the `common.vhd` bus.

4.2.2.1 Instruction Decode Control Unit

The control path of the ID stage is managed by the Instruction Decode Control Unit (`id_control_unit.vhd`), a combinational decoder that translates the instruction's 7-bit opcode field into control signals for the rest of the processor.

Opcode Decoding: The unit maps standard RISC-V opcodes to their corresponding operation classifications (type `common.vhd`) and target execution units (type `common.vhd`). This includes setting register write enables, memory read/write strobes, operand multiplexer selectors, and writeback data sources.

Landing Pad (OP_LPAD) Decoding Override: In standard RISC-V architectures, the opcode "0010111" maps to the Add Upper Immediate to PC (AUIPC) instruction. However, to support forward-edge Control-Flow Integrity (CFI) via the Zicfilp extension, the control unit implements a custom decoding override. If the landing pad expected flag (`i_e1p`) is active and the destination register field of the instruction is zero (indicating register `x0`), the control unit overrides normal decoding. It classifies the instruction as a landing pad (OP_LPAD), routes Operand A to read from the register file, routes Operand B to use the immediate label, and disables register write updates.

Shadow Stack System Command Decoding: The control unit decodes system instructions (opcode "1110011") to identify custom shadow stack commands under the Smcfiss extension (Funct3 = "100"). A write instruction (Shadow Stack Push) enables memory write controls, routing the register inputs to memory. A read instruction (Shadow Stack Pop) checks the destination register field. If it is zero, the control unit bypasses the register file write path but enables memory read controls to retrieve the saved return address. If it is non-zero, it enables the register file write path to restore the return address to a destination register.

CSR and System Instruction Decoding: For standard CSR instructions, the control unit selects the operand sources based on the instruction type. Immediate CSR instructions route the zero-extended unsigned immediate field as Operand A, while register CSR instructions route the general register source.

4.2.2.2 Register File

The Register File (`register_file.vhd`) is a 32-entry general-purpose register file (x0–x31) implementing the integer registers of the RV32I ISA.

Synchronous Writes: Register writes occur on the rising edge of the global clock, provided that the write enable signal is active and the destination register is not x0. Register x0 is hardwired to zero, and writes targeting it are ignored.

Asynchronous Reads: The module provides two independent, asynchronous combinational read ports. The source registers are read combinatorially to allow the Execute stage to receive operand data in the same clock cycle.

Internal Write-During-Read Bypassing: When a read operation and a write operation target the same register in the same clock cycle, reading the register array directly would return stale data, stalling the pipeline. To resolve this, the register file implements internal bypass logic. If the write address matches a read port address and is not register zero, the register file routes the write data directly to the corresponding read output port. This internal forwarding resolves data hazards within the register file without inserting pipeline stall cycles.

4.2.2.3 Immediate Reconstruction Unit

The Immediate Reconstruction Unit (`immediate_reconstruct_unit.vhd`) is a combinational module that extracts and sign-extends immediate fields from the instruction data.

RISC-V distributes immediate bits across different fields in the instruction to align source register indices to fixed positions, reducing decoding logic. The reconstruction unit parses instruction bits 6 down to 2 to reconstruct the appropriate immediate format:

U-Type immediates (LUI, AUIPC): Extracts the upper 20 bits and appends 12 trailing zeros.

J-Type immediates (JAL): Reorders a 20-bit signed offset and left-shifts it by one bit (appending a trailing zero) for PC-relative jump target calculations.

B-Type immediates (Branch): Reorders a 12-bit signed offset and left-shifts it by one bit (appending a trailing zero) for PC-relative branching.

I-Type immediates (JALR, Loads, Arithmetic immediates): Extracts a 12-bit signed immediate and sign-extends the upper bits to 32 bits.

S-Type immediates (Stores): Reconstructs a 12-bit signed immediate from split fields in the instruction, sign-extending the upper bits.

CSR immediates: Extracts the 12-bit immediate field (representing the CSR address or system parameter) and zero-extends the upper bits.

4.2.2.4 Control-Flow Integrity (CFI) Overrides in the Decode Stage

To support the core's control-flow hardening features, the top-level wrapper implements specific overrides during the decode stage.

Landing Pad Register Override: Under the JOP compiler convention for the Zicfilp extension, the expected target landing pad label is stored in register x7 (t2) during indirect jumps. When the pipeline indicates that a landing pad instruction is expected (`elp_active = '1'`), the Decode stage overrides the first source register address (`rs1_addr`), hardcoding it to "00111" (register x7). This forces the Register File to read the target label from register x7, routing it to the Execute stage for comparison against the landing pad immediate label.

Fault Propagation and Bubble Insertion: If the incoming instruction contains an active fault tag (such as an access fault from the Fetch stage), the Decode stage overrides the decoded control signals. It clears memory read, memory write, and register write enables, and forces the operation type to a standard R-type instruction. This inserts a passive instruction bubble into the pipeline to prevent the faulted instruction from modifying the processor state before the fault tag is resolved.

4.2.3 Execution (EX) Stage

Serving as the computational heart and primary control-flow resolver of the eSC-V processor, the Execution (EX) stage is structurally encapsulated within the highly modular `execution_stage.vhd` entity. While the preceding IF and ID stages are primarily concerned with fetching and parsing instruction bits sequentially, the EX stage relies on dense, parallel combinational logic to physically compute mathematical results, evaluate complex branching conditions, and actively modify the machine's architectural state.

To manage this micro-architectural complexity without creating a monolithic bottleneck, the top-level wrapper instantiates six interconnected subsidiary hardware modules:

- The Execution Control Unit (`ex_control_unit.vhd`)
- The Arithmetic Logic Unit (ALU) (`alu.vhd`)
- The Branch Adder (`branch_adder.vhd`)

- The Branch Control Unit (`branch_control_unit.vhd`)
- The Control and Status Register (CSR) Unit (`csr_unit.vhd`)
- The Forwarding Unit (`forwarding_unit.vhd`)

The top-level wrapper coordinates the interfaces between these modules. It ingests decoded controls and operands from the ID stage via the `common.vhd` bus and outputs calculation results, memory write-data, and exception flags to the MEM stage via the `common.vhd` bus. In parallel, it communicates redirections (such as branch-taken destinations and exception trap targets) to the Instruction Fetch stage via the `common.vhd` interface.

4.2.3.1 Execution Control Unit

While the ID stage performs the initial, high-level instruction decode, the EX stage requires highly specific, localized micro-architectural routing signals to coordinate its internal computational components. The local decoder (`ex_control_unit.vhd`) fulfills this critical role by acting as a secondary decoding block. It systematically ingests the generalized operation type (`i_opr_type` of type `common.vhd`), the `funct3` field, the `funct12` field, and the source register data (`i_src_a_data`) passed down from the ID/EX pipeline bus.

By analyzing these fields alongside the requested register write enables, the EX Control Unit dictates the operational parameters for both the ALU and the CSR hardware. It explicitly sets the ALU command (type `common.vhd`) to define the precise mathematical operation the ALU must execute for the current cycle. For instance, load upper immediate operations map to a pass-through command, branch and landing pad operations map to subtraction, and register-to-register or register-to-immediate instructions map to their corresponding arithmetic, logic, or shift operators by analyzing the function fields.

Additionally, the unit generates the CSR command (type `common.vhd`) and asserts write strobes to orchestrate reads, writes, or atomic bitwise manipulations within the CSR hardware. During this process, it implements the standard RISC-V optimization where writing to a CSR is bypassed if the source register is `x0` during a set or clear instruction, preventing unnecessary write-cycles.

Most importantly for system security, this control unit actively evaluates the instruction context to trigger system traps by emitting specific trap signals (type `common.vhd`) when environment calls, software breakpoints, machine returns, or illegal instructions are decoded. This hardware-level trap generation ensures that illegal operations, invalid architectural states, or Control-Flow Integrity (CFI) violations are immediately caught and flagged before they can advance to the Memory stage.

4.2.3.2 Arithmetic Logic Unit (ALU)

The ALU (`alu.vhd`) serves as the primary 32-bit mathematical engine of the eSC-V core, explicitly designed to execute the computational requirements of the RV32I base instruction set. It performs operations dynamically based on the opcode provided by the EX Control Unit.

To optimize silicon area and conserve valuable FPGA resources, the ALU does not duplicate hardware for every mathematical operation; instead, it utilizes a highly efficient unified adder-subtractor architecture. Rather than deploying separate physical logic gates for subtraction and comparisons, the

module relies on two's complement arithmetic. The ALU checks the incoming opcode, and if subtraction or comparisons are requested, it dynamically generates an internal carry-in bit of one and bitwise inverts Operand B. The unit then performs a signed 33-bit addition of Operand A, the modified Operand B, and the carry-in bit to generate a 32-bit addition result and carry flags.

The ALU physically generates four standard hardware flags (Zero, Carry, Negative, Overflow) and executes the following full matrix of operations:

Arithmetic: Standard integer addition and subtraction.

Logical: Bitwise operations including bitwise AND, OR, and XOR.

Shifts: A barrel shifter extracts the lowest five bits of Operand B to determine the shift amount. The unit then performs Logical Left Shift, Logical Right Shift, or Arithmetic Right Shift (which preserves the sign bit during the shift).

Comparisons: Set Less Than for signed integers and Set Less Than Unsigned. Signed comparison evaluates the Negative flag $\oplus$ Overflow flag, while unsigned comparison checks if the Carry flag is deasserted.

Pass-Through: A dedicated copy command, which bypasses arithmetic logic to forward raw immediate values directly to the output bus, primarily utilized for executing the Load Upper Immediate instruction.

4.2.3.3 Branch Resolution Architecture

Control-flow mutations—such as conditional branches and absolute jumps—are fully resolved within the EX stage using two specialized, concurrent hardware modules: the Branch Adder and the Branch Control Unit. By splitting this logic, the core executes address generation and condition evaluation in parallel.

1. The Branch Adder (`branch_adder.vhd`)

This module is a dedicated arithmetic unit tasked exclusively with computing the 32-bit target address for jumps and branches. It performs a signed, 32-bit addition combining a base address (which is either the current Program Counter or the source register value) and a sign-extended immediate offset. By completely abstracting this target calculation away from the main ALU, the core can securely compute the target destination concurrently while the ALU is simultaneously calculating the branch condition itself. The Branch Adder generates the branch address for conditional branches, Jump and Link (PC + offset), Jump and Link Register (RS1 + offset), and Add Upper Immediate to PC (PC + upper immediate).

To enforce architectural rules, the wrapper checks the target addresses. For jump instructions using PC-relative offsets, it asserts a misalignment fault if the computed target address does not align to a 4-byte boundary. For register-relative jumps (like JALR), the misalignment check inspects the ALU result to ensure that bit 1 is not set, generating a misalignment fault if a violation is detected.

2. The Branch Control Unit (`branch_control_unit.vhd`)

Operating in strict parallel with the Branch Adder, this unit dictates if a conditional branch should

actually be taken. It accomplishes this by actively evaluating the raw hardware flags generated by the main ALU (specifically the Zero, Carry, Negative, and Overflow flags) against the function selector field of the current branch instruction.

It explicitly decodes the following RV32I conditional branch criteria:

- **Branch if Equal:** Validated if the ALU's Zero flag is set.
- **Branch if Not Equal:** Validated if the ALU's Zero flag is clear.
- **Branch if Less Than (signed):** Validated by calculating $\text{Negative} \oplus \text{Overflow}$.
- **Branch if Greater or Equal (signed):** Validated by calculating the logical negation of $\text{Negative} \oplus \text{Overflow}$.
- **Branch if Less Than (unsigned):** Validated if the Carry flag is clear.
- **Branch if Greater or Equal (unsigned):** Validated if the Carry flag is set.

Crucially, the unit only authorizes a pipeline redirection by setting the branch-taken signal to one if the evaluated condition is true AND the overarching branch-active signal is asserted, guaranteeing that arbitrary ALU operations do not trigger ghost branches.

4.2.3.4 Control and Status Register (CSR) Unit

Implemented within `csr_unit.vhd`, this synchronous module manages the internal architectural state of the core beyond the standard integer register file. Governed by the global system clock and synchronous reset, the CSR unit serves as the structural backbone of the eSC-V's ZiCSR extension integration.

The CSR unit physically implements the following registers:

- **Machine Information (Read-Only):** The vendor ID register and the architecture extension register (which identifies the core as a 32-bit base integer execution environment).
- **Machine Status (mstatus):** Manages the machine interrupt enable bit and the machine previous interrupt enable bit.
- **Trap Handling Support:** Registers for the trap vector base address (mtvec), temporary scratch data (mscratch), machine exception program counter (mepc), machine trap cause (mcause), machine trap value (mtval), and machine interrupt enable flags (mie).
- **Hardware Performance Counters:** Standard 32-bit counters for active clock cycles (mcycle) and instructions retired (minstret).
- **Physical Memory Protection (PMP):** The PMP configuration register (pmpcfg0) and four address registers (pmpaddr0 through pmpaddr3).
- **Custom Security Registers:** The shadow stack pointer register (ssp), and the machine security configuration register (mseccfg) containing landing pad and shadow stack enable bits.

1. Indirect PMP Access (Smpmpind Extension)

To minimize address decoding complexity, the CSR unit implements the Smpmpind extension. The software selects a PMP register index (zero to three) by writing to the machine indirect select register (miselect). When reads or writes are subsequently performed on the indirect registers (mireg and mireg2), the unit uses the selector index to access the physical registers. Accesses to mireg read or modify the selected PMP address register (pmpaddr0-3). Accesses to mireg2 read or modify the corresponding PMP configuration byte and lock execution configuration bits.

2. Trap Processing and Exception Updates

When a trap is asserted, the unit synchronously saves the program counter of the trapping instruction to the machine exception program counter (mepc), the trap cause to mcause, and the trap-specific value to mtval. Simultaneously, the machine interrupt enable (mie) bit is saved into the machine previous interrupt enable (mpie) bit, and the active interrupt enable is cleared to zero to temporarily disable interrupts. Conversely, executing a machine mode return (MRET) restores the interrupt enable from the previous interrupt enable state and sets the previous state to one.

4.2.3.5 Forwarding Unit

To resolve data dependencies and prevent pipeline stalling, the EX stage integrates the asynchronous forwarding_unit.vhd. When an instruction in the EX stage requires data from a register that is currently being modified by an older instruction located further down the pipeline (in the MEM or WB stages), simply fetching data from the ID stage's register file will yield a stale, mathematically incorrect value.

The Forwarding Unit logic continuously evaluates the source register addresses of the current EX instruction. It actively compares these addresses against the destination registers of the instructions currently occupying the Memory and Writeback pipeline registers:

Operand A Bypassing: The unit checks if the instruction in the Memory stage is configured to write to the register file, if its destination register is not register zero, and if that destination register matches the source register address of the instruction in the Execute stage. If all three conditions are satisfied, the unit select signal routes the forwarded value from the Memory stage. If this condition is not met, the unit checks the same criteria against the Writeback stage. If a match occurs there, it routes the forwarded value from the Writeback stage. Otherwise, it defaults to the value read from the Register File.

Operand B Bypassing: Bypassing is evaluated using the same logic matching the second source register address.

CSR Forwarding: Bypassing is also extended to Control and Status Registers. If an instruction in the Memory or Writeback stage writes to a CSR, and its target address matches the CSR address requested in the Execute stage, the Forwarding Unit directs a multiplexer to bypass the raw register output and route the updated CSR data forward. This prevents pipeline bubbles during consecutive CSR read/write operations.

4.2.3.6 Hardware Control-Flow Integrity (CFI) Hardening

The EX stage is the primary enforcement point for the core's Control-Flow Integrity (CFI) security extensions.

1. Forward-Edge CFI (Zicfilp Landing Pads)

Forward-edge CFI checks that indirect jumps (e.g. JALR) target a valid landing pad instruction (lpad), preventing Jump-Oriented Programming (JOP) attacks.

- **Enforcement Logic:** If the landing pad check is enabled globally in the security configuration and the pipeline is expecting a landing pad instruction, the unit inspects the incoming operation type. If the operation is not a landing pad instruction, it raises a landing pad fault. Additionally, it checks the ALU status flags to verify that the landing pad label matches the expected register value. If a mismatch is detected, a fault is flagged, preventing the instruction from modifying memory or registers.
- **ELP State Propagation:** Upon executing an indirect jump, the unit checks the source register address. If the register is not one of the designated link registers—specifically the return address (ra/x1), or the temporary registers (t0/x5 and t2/x7)—the Execute stage asserts a redirection signal to the Instruction Fetch stage. This sets the expected landing pad state, forcing the next fetched instruction to be a landing pad.

2. Backward-Edge CFI (Smcfiss Shadow Stack)

The shadow stack protects function return addresses from buffer overflow attacks (Return-Oriented Programming).

- **Enforcement Logic:** The shadow stack pointer is stored in the ssp CSR, isolating it from the program stack pointer in the general register file. When a shadow stack push is decoded, the Execute stage calculates the updated shadow stack pointer by subtracting four bytes from the current pointer value. This updated value is written back to the shadow stack pointer register and routed as the target address to the Memory stage. Simultaneously, the return address is passed along the memory bus. For a shadow stack pop, the pointer is incremented by four bytes.
- **Memory Redirection:** The stage asserts a dedicated shadow stack control line to instruct the memory controller to redirect the access to the protected memory region where the shadow stack resides.

3. Redirections and Pipeline Synchronization

If any instruction fault is detected or a branch/jump redirection occurs, the EX stage asserts a pipeline flush to clear invalid instructions from the Fetch and Decode stages.

When configuration registers that modify memory access rules or security enforcement (such as the PMP registers or the security configuration register) are written to, it can create a race condition where subsequent instructions execute under old rules. To resolve this, writes to these registers trigger a self-redirection trap, forcing the pipeline to flush and reload the following instructions under the newly applied security configurations.

4.2.4 Memory Access (MEM) Stage

Serving as the data interface to the memory subsystem, the Memory Access (MEM) stage is structurally encapsulated within the `memory_stage.vhd` entity. While computational instructions simply pass through this stage, load, store, and shadow stack instructions interact with the memory controller and physical peripherals.

The MEM stage wrapper is positioned between the Execute and Writeback stages. It receives control vectors, ALU calculation results (which represent memory addresses), and write data from the Execute stage via the `common.vhd` bus, performs address alignment checks, generates memory write enables and byte enable masks, and propagates the read data and status flags to the Writeback stage via the `common.vhd` bus.

4.2.4.1 Memory Access Alignment & Hardware Misalignment Checking

To simplify the data memory controller and maximize access speed, the eSC-V core enforces strict memory alignment rules in hardware. If a software program attempts an unaligned memory access, the MEM stage detects this combinational mismatch and blocks the transaction.

The alignment check evaluates the target memory address based on the transfer size specified in the instruction:

- **Word Accesses (32-bit):** The target address must align to a 4-byte boundary. The hardware checks that the lowest two bits of the address are zero. If they are not, a misalignment flag is raised.
- **Halfword Accesses (16-bit):** The target address must align to a 2-byte boundary. The hardware checks that the lowest bit of the address is zero. If it is not, a misalignment flag is raised.
- **Byte Accesses (8-bit):** Byte accesses are always aligned, so no check is performed.

If a misalignment is detected during a read operation, the instruction's fault tag (type `common.vhd`) is updated to a Load Misaligned fault. If it occurs during a write operation, the fault tag is updated to a Store Misaligned fault.

4.2.4.2 Physical Memory Protection (PMP) Fault Integration

The MEM stage integrates with the core's Physical Memory Protection (PMP) unit. The PMP unit monitors all data addresses and generates a PMP fault signal if an instruction attempts to access a protected memory region without sufficient privileges.

If the PMP unit asserts a fault signal for a memory access, the MEM stage checks the transaction type:

- **Memory Reads:** An access violation during a read operation updates the instruction's fault tag to a Load Access Fault.
- **Memory Writes:** An access violation during a write operation updates the instruction's fault tag to a Store Access Fault.

- **Shadow Stack Operations:** Because the shadow stack is a critical security region, PMP faults during shadow stack instructions (whether reads or writes) are automatically treated as Store Access Faults, triggering a system trap.

4.2.4.3 Memory Store Byte-Lane Alignment & Write Data Replication

Data memory in the eSC-V is organized as a 32-bit wide memory space. When executing store instructions that write less than a full 32-bit word, the data must be aligned to the correct byte lanes on the 32-bit data bus, and a byte enable mask must be generated to write only to the targeted bytes.

To achieve this, the MEM stage implements a replication and masking process:

- **Word Stores (SW):** The write data from register rs2 is routed directly to the memory bus, and the byte enable mask is set to enable all four bytes.
- **Halfword Stores (SH):** The lower 16 bits of the write data are replicated across both the lower and upper halves of the 32-bit write bus. If the lowest two bits of the target address indicate the lower halfword, the byte enable mask is set to write bytes 0 and 1. If they indicate the upper halfword, the mask is set to write bytes 2 and 3.
- **Byte Stores (SB):** The lower 8 bits of the write data are replicated across all four byte lanes of the 32-bit write bus. The lowest two bits of the address are decoded to enable only the targeted byte lane (byte 0, 1, 2, or 3) in the byte enable mask.
- **Shadow Stack Writes:** Shadow stack push operations write return addresses as full 32-bit words, so the byte enable mask is set to enable all four bytes.

4.2.4.4 Fault Blocking & Writeback Prevention

To prevent unauthorized memory modifications or state corruption, the MEM stage implements hardware write-protection.

If the instruction's incoming fault tag is not marked as valid, or if the stage detects a misalignment or PMP fault:

- **Memory Write Blocking:** The memory write enable signal and the memory transaction valid signal are forced to zero. This hardware override prevents invalid instructions from modifying the data memory or peripheral states.
- **Writeback Suppression:** The register write enable and CSR write enable signals propagated to the Writeback stage are forced to zero. This ensures that a faulted instruction cannot modify the general-purpose register file or system configuration registers.

4.2.4.5 Shadow Stack Pop Datapath Bypass

During a shadow stack pop instruction (which reads a return address from memory), the popped return address must be compared against the program return address to detect stack corruption.

To support this check, the MEM stage implements a datapath bypass. When a shadow stack pop instruction is active, the expected return address (passed down from the Execute stage in the second register data field) is routed to the writeback register data bus. In the Writeback stage, the actual return address loaded from the shadow stack memory is compared against this expected value. If a mismatch is detected, a shadow stack fault is raised, trapping the CPU.

4.2.5 Writeback (WB) Stage

Serving as the final stage of the eSC-V pipeline, the Writeback (WB) stage is structurally encapsulated within the `writeback_stage.vhd` entity. While preceding stages calculate results and fetch memory data, the WB stage is responsible for committing these modifications to the register file, handling precise exceptions, and retiring instructions.

The WB stage receives control signals, ALU results, and memory access states from the Memory stage via the `common.vhd` bus, receives raw read data from the data memory interface, and generates register writeback feedback (`common.vhd`) for the Decode stage and CSR/exception feedback (`common.vhd`) for the Execute stage.

4.2.5.1 Precise Exception & Trap Generation

To maintain precise exception handling, faults are not processed immediately in the pipeline stage where they occur. Instead, exception markers (type `common.vhd`) are attached to the instruction and propagated through the pipeline. When a faulted instruction reaches the WB stage, it is guaranteed to be the oldest active instruction, and the stage commits the trap:

- **Trap Activation:** The stage asserts the trap signal, saving the PC of the faulted instruction to the machine exception program counter (`mepc`).
- **Exception Code Mapping:** The fault tag is translated into a standard RISC-V Exception Code (`mcause`) and the offending address/value is loaded into the machine trap value (`mtval`):
 - **Fetch Access Fault:** Maps to `mcause` code 1 (Instruction Access Fault), loading the invalid PC into `mtval`.
 - **Landing Pad Fault (Zicfilp violation):** Maps to `mcause` code 18 (landing pad violation).
 - **Target Address Misalignment:** Maps to `mcause` code 0 (Instruction Address Misaligned) for jumps/branches, loading the misaligned target address into `mtval`.
 - **Data Address Misalignment:** Maps to `mcause` code 4 (Load Address Misaligned) or code 6 (Store Address Misaligned), loading the misaligned address into `mtval`.
 - **Data Memory Access Violations:** Maps to `mcause` code 5 (Load Access Fault) or code 7 (Store Access Fault), loading the target address into `mtval`.
 - **Software Breakpoints:** Maps to `mcause` code 3 (Breakpoint), loading the PC into `mtval`.
 - **Environment Calls (ECALL):** Maps to `mcause` code 11 (Environment Call from M-mode).
 - **Machine Mode Return (MRET):** Asserts the machine return signal to instruct the CSR unit to restore the previous interrupt state.

4.2.5.2 Shadow Stack Return Address Validation

The WB stage serves as the final validation check for the hardware-enforced shadow stack under the Smcfiss extension, preventing Return-Oriented Programming (ROP) exploits.

When a shadow stack pop instruction reaches the WB stage, the actual return address loaded from the shadow stack memory is compared against the expected return address (the value read from the standard link register, which was routed to the WB stage via the Memory stage).

If the loaded return address does not match the expected value, the validation logic intercepts the writeback. It overrides the fault tag to a Shadow Stack Fault, sets the exception cause (mcause) to code 18 (denoting a CFI violation), loads the instruction's PC into the exception program counter (mepc), and asserts the trap signal. This halts the processor before the corrupted return address can be used by a return instruction.

4.2.5.3 Memory Load Data Alignment & Extension

When a load instruction reads from memory, the data memory interface returns a raw 32-bit word. The WB stage extracts the targeted byte or halfword based on the address alignment and applies the correct sign or zero extension:

- **Byte Extraction & Extension:** The stage checks the lowest two bits of the memory address to isolate the targeted byte lane from the 32-bit data word. For signed byte loads (LB), the 8-bit value is sign-extended to 32 bits. For unsigned byte loads (LBU), the value is zero-extended.
- **Halfword Extraction & Extension:** The stage checks the second bit of the address to isolate the lower or upper 16-bit halfword. For signed halfword loads (LH), the 16-bit value is sign-extended to 32 bits. For unsigned halfword loads (LHU), it is zero-extended.
- **Word Loads:** Standard load word (LW) and shadow stack pop instructions bypass the alignment multiplexers, routing the full 32-bit memory word without modification.

4.2.5.4 Writeback Data Multiplexing & Forwarding Feedback

The WB stage uses a multiplexer to select the source of the register writeback data:

- **ALU Results:** Routes the calculated result from the Execute stage.
- **Memory Load Data:** Routes the aligned and extended memory read data.
- **Return Addresses:** Routes the next-sequential program counter (PC + 4) for Jump and Link (JAL/JALR) instructions.

This selected data is routed back to the Decode stage register file to be written on the next clock edge. Simultaneously, the data is routed back to the Execute stage forwarding multiplexers, resolving data hazards for subsequent instructions.

4.2.5.5 Critical CSR Write Handling & Pipeline Synchronization

Writing to critical system registers that alter security or memory protection rules (such as the PMP configurations, PMP addresses, or the security configuration register) can introduce race conditions if subsequent instructions execute before these new rules are active.

To prevent this:

- **Critical CSR Detection:** The WB stage checks if the instruction is writing to any critical CSR address (including PMP configurations, PMP addresses, or the `mseccfg` register).
- **Redirection Flush:** If a critical CSR write is detected, the stage asserts a critical CSR flag and triggers a pipeline flush.
- **Pipeline Synchronization:** The pipeline flush clears all instructions currently in the Fetch, Decode, Execute, and Memory stages, and redirects the Instruction Fetch stage to reload from the next sequential PC. This guarantees that all subsequent instructions are fetched and executed under the newly committed security configuration.
- **Counter Increment:** If the instruction retires successfully without fault, the stage asserts the instruction retirement signal, which is fed back to the CSR unit to increment the retired instructions counter (`minstret`).

4.2.6 Hazard Detection Unit

Acting as the coordinator of pipeline execution flow, the Hazard Detection Unit is structurally implemented within the asynchronous `hazard.detection.unit.vhd` module. In a pipelined CPU, data hazards occur when an instruction requires the result of a preceding instruction that has not yet committed its values. While most data hazards in the eSC-V core are resolved dynamically using forwarding paths, load-use hazards cannot be resolved by forwarding alone.

A load-use hazard arises when an instruction in the Instruction Decode (ID) stage requires data from a load instruction (such as LW, LH, or LB) currently in the Execution (EX) stage. Because the load data must be fetched from data memory, it is not available until the end of the Memory Access (MEM) stage. This introduces a one-cycle delay that forwarding cannot eliminate. The Hazard Detection Unit resolves this hazard by combinationally monitoring the instruction stream and stalling the pipeline when necessary.

4.2.6.1 Load-Use Hazard Condition & Combinational Evaluation

The Hazard Detection Unit operates combinationally, evaluating source and destination registers on each clock cycle to detect data hazards. It compares the source register addresses of the instruction in the Decode stage against the destination register address of the instruction in the Execute stage.

A load-use hazard is detected when all of the following conditions are met:

1. **Load Instruction in EX:** The memory read enable signal from the instruction in the Execute stage is active, indicating a load operation is in progress.

2. **Valid Destination Register:** The destination register address of the load instruction is not register zero ($x0$). In the RISC-V ISA, register zero is hardwired to zero and cannot be modified, so writes to it do not trigger hazards.
3. **Register Address Match:** The destination register address of the load instruction in the Execute stage matches either the first source register address ($rs1$) or the second source register address ($rs2$) of the instruction in the Decode stage.

When this combinational check evaluates to true, the unit asserts the pipeline stall output signal.

4.2.6.2 Pipeline Stall Coordination & Bubble Insertion

When the stall output signal is asserted, it controls the pipeline registers in the top-level core wrapper:

- **Freezing the Front-End:** The stall signal disables write updates to the Program Counter register in the Fetch stage and the IF/ID pipeline register. This holds the instruction in the Fetch stage and keeps the dependent instruction in the Decode stage for an additional clock cycle.
- **Bubble Insertion:** Simultaneously, the stall signal flushes the ID/EX pipeline register, resetting it to its default NOP state. This inserts a passive instruction bubble into the Execute stage, preventing the dependent instruction from executing with stale data.
- **Hazard Resolution:** During the stall cycle, the load instruction proceeds from the Execute stage to the Memory Access stage, where the data is retrieved from memory. On the subsequent clock edge, the stall is deasserted. The Forwarding Unit in the Execute stage then bypasses the newly retrieved data from the Memory Access stage directly to the ALU inputs, allowing the dependent instruction to execute without stalls or data corruption.

4.2.7 Physical Memory Protection (PMP) Unit

Residing as a global security filter outside the pipeline stages, the Physical Memory Protection (PMP) unit is structurally implemented in `pmp_unit.vhd`. The PMP unit is responsible for monitoring all instruction fetch addresses and data memory access addresses combinationally. It compares these addresses against the configured PMP address and configuration registers to enforce access permissions.

To protect critical system states, the PMP unit evaluates access rules based on three criteria: the active execution privilege mode, the target memory region permissions, and whether the memory request is generated by standard instructions or hardware-hardened security operations.

4.2.7.1 Address Matching Modes

The PMP unit supports four independent memory protection regions, using PMP address registers (`pmpaddr0` through `pmpaddr3`) and configuration fields in the PMP configuration register (`pmpcfg0`). The address checking logic matches target addresses using one of four configurable modes:

- **Disabled Mode (00):** The region is inactive, and no address comparison or privilege checking is performed.

- **Top of Range (TOR) Mode (01):** The target address matches if it falls within the range from the previous PMP address register value (inclusive) to the current PMP address register value (exclusive). For region zero, the lower bound defaults to address zero.
- **Naturally Aligned 4-Byte (NA4) Mode (10):** Matches a single 4-byte memory boundary.
- **Naturally Aligned Power-of-Two (NAPOT) Mode (11):** Matches power-of-two sized regions from 8 bytes up to several gigabytes. The region size is determined by the position of the lowest zero bit in the PMP address register, which the hardware uses to generate a bitmask to match address boundaries combinationally.

4.2.7.2 Shadow Stack Region Hardening

To prevent attackers from modifying return addresses (Return-Oriented Programming), the core implements the `Smpmpind` and `Smcfiss` extensions. The PMP unit uses these configuration bits to establish a secure shadow stack region:

- **Shadow Stack Classification:** A memory region is classified as a Shadow Stack region if the custom execution enable bit is set and the standard configuration permission bits match the binary pattern for write-only access.
- **Access Control Policies:** Once a region is identified as a shadow stack region, the PMP unit enforces the following rules:
 1. **Instruction Execution:** Any attempt to fetch instructions from this region is blocked.
 2. **Memory Writes:** Writes are restricted; they are only allowed if the write is requested by a hardware shadow stack instruction (such as a function call push). Standard software store instructions attempting to write to this region are blocked.
 3. **Memory Reads:** Reads are permitted to allow return address verification.
- **Access Isolation:** If a hardware shadow stack instruction attempts to access a standard memory region (outside the designated shadow stack region), the transaction is blocked. This ensures that shadow stack operations cannot overwrite standard program data.

4.2.7.3 Transaction Interception & Address Overriding

The PMP unit operates combinationally on both memory interfaces, intercepting unauthorized transactions before they reach the memory controller:

- **Instruction Fetch Violations:** If a fetch address violates PMP rules, the unit asserts a fetch fault signal and overrides the output instruction address to all ones (`0xFFFFFFFF`). This blocks the unauthorized instruction from being fetched.
- **Data Access Violations:** If a data read or write address violates PMP rules, the unit asserts a memory fault signal, deasserts the memory write enable signal to block writes, and overrides the output data address to all ones.

- **Trap Propagation:** The fault signals are routed back to the Instruction Fetch and Memory Access stages, where they are captured as access fault tags and propagated down the pipeline to trigger precise exception traps in the Writeback stage.

4.3 Unified Memory Controller & BRAM Interface

Implementing the core's memory subsystem, the Unified Memory Controller is structurally defined within `unified_memory_unit.vhd`. The memory unit provides Harvard architecture semantics (separate instruction and data interfaces) to the processor core while mapping both ports to a single underlying memory array.

By using a dual-port memory structure, the core can fetch instructions and perform data read or write operations simultaneously in a single clock cycle, preventing memory access stalls during execution.

4.3.1 Dual-Port Memory Access & Interface Semantics

The memory unit is word-addressable (storing 32-bit words) but provides a byte-addressed interface to match the RISC-V specification. It ignores the lowest two address bits of the incoming address signals, assuming word alignment, and uses bits 29 down to 2 as the word index into the storage array.

- **Instruction Port:** Implements a read-only interface that fetches a 32-bit instruction on every clock cycle. If the program counter falls outside the valid memory range, the port returns a standard NOP instruction to prevent undefined execution.
- **Data Port:** Implements a read/write interface that supports byte-granular writes. A 4-bit byte-enable mask indicates which bytes of the 32-bit word should be updated, allowing byte and half-word store instructions (SB and SH) to update memory without requiring read-modify-write operations.
- **Synchronous Read Latency:** Reads on both ports are synchronous, with a 1-cycle read latency. The data requested in the current clock cycle is registered and output on the subsequent clock edge, matching the hardware behavior of physical block RAMs and ensuring timing closure during FPGA synthesis.

4.3.2 Block RAM (BRAM) Inference & Simulation Modes

To optimize compilation and performance, the memory unit supports two compile-time operational modes selected via generic parameters:

- **Simulation Mode:** Designed to reduce overhead in simulation tools. The memory array is represented as a VHDL shared variable, reducing simulation memory usage and accelerating compile times.
- **Synthesis Mode:** Designed for physical FPGA deployment. The memory array is represented using VHDL signals. The architecture includes a `ram_style` attribute set to "block_ram", which forces the synthesis tools to map the memory array directly to the block RAM blocks of the target

FPGA (such as the Tang Primer 20K). The byte-enable write logic is structured to match the hardware byte-lane write circuits of FPGA block RAMs, ensuring block RAM inference.

4.3.3 Hexadecimal Initialization & Bootloader Considerations

For execution, the memory must contain the program binary. The memory unit initializes its contents during compile time:

- **Hexadecimal Initialization:** The architecture uses an impure VHDL initialization function that reads a compiled hexadecimal program file (`code.hex`) during compilation. The function reads the hexadecimal words sequentially and hardcodes the binary code directly into the Block RAM logic arrays during synthesis.
- **Simplicity and Bitstream Integration:** This approach simplifies program loading during development, as the software binary is embedded directly into the synthesized FPGA bitstream and loaded automatically upon power-up.
- **Future Bootloader Improvements:** While hardcoding the binary code is used for simplicity in the current design, implementing a dynamic hardware bootloader is planned as a future improvement. This bootloader will interface with the UART peripheral, allowing developers to load program binaries into the Block RAM memory dynamically after the FPGA has been programmed.

4.4 UART Peripheral

Serving as the primary serial I/O peripheral for communicating with external systems, the UART transceiver is structurally implemented in `uart.vhd`. The transceiver provides full-duplex communication over serial lines, supporting an 8-bit data frame format with no parity and one stop bit (8N1 configuration).

The UART hardware design is divided into three functional blocks: a clock divider (baud rate generator), a serial transmitter state machine, and a serial receiver state machine with noise-filtering and synchronization features.

4.4.1 Baud Rate Generator & Clock Divider

To synchronize serial communication, the UART unit divides the high-frequency system clock down to the selected bit period. The module takes two parameters during synthesis: the system clock frequency (defaulting to 27 MHz) and the target baud rate (defaulting to 115200 bits per second).

Using these parameters, the hardware calculates a constant representing the number of system clock cycles per serial bit period:

$$\text{Clock Cycles per Bit} = \frac{\text{System Clock} + \frac{\text{Baud Rate}}{2}}{\text{Baud Rate}}$$

This constant is used by counters in the transmitter and receiver state machines to divide the clock, ensuring accurate timing synchronization for serial transmission and reception.

4.4.2 Transmitter (TX) State Machine

The transmitter state machine converts parallel data bytes from the processor into a serial stream. It sequences through four states:

- **Idle:** The transmitter asserts a ready flag to indicate it can accept new data. When a transmit request is received, the unit captures the data byte into an internal shift register, deasserts the ready flag to block incoming data, and transitions to the start bit state.
- **Transmit Start:** The transmitter forces the serial output line low for one bit period to signal the start of a frame.
- **Transmit Data:** The unit shifts out the 8 data bits sequentially, starting with the least significant bit (LSB) first. Each bit is held on the serial output line for one bit period before the shift register updates.
- **Transmit Stop:** The transmitter forces the serial output line high for one bit period to transmit the stop bit, then returns to the Idle state and reasserts the ready flag.

4.4.3 Receiver (RX) State Machine & Metastability Filter

Because the incoming serial RX line is asynchronous to the processor's internal clock, reading it directly could cause metastability in the receiver logic. The receiver implements synchronization and noise filtering to ensure reliable reception:

- **Metastability Filter:** The raw serial input line passes through a two-stage flip-flop synchronizer clocked by the system clock. This synchronizer filters out metastability and aligns the asynchronous transitions with the processor's clock domain.
- **Receiver State Machine:** The receiver sequences through four states to reconstruct the serial stream:
 1. **Idle:** The receiver monitors the synchronized RX line for a falling edge, which indicates a start bit.
 2. **Verify Start:** To filter out line noise and glitches, the receiver waits for half a bit period and samples the line again. If the line is still low, the start bit is verified, and the receiver transitions to the data state. If the line has returned high, the transition is ignored as a glitch, and the receiver returns to the Idle state.
 3. **Receive Data:** The receiver samples each of the 8 data bits at its mid-bit time (the center of each bit period), where the signal is most stable. The sampled bits are shifted into an internal register.
 4. **Receive Stop:** The receiver waits for the stop bit period, asserts the receiver-new flag for one clock cycle to signal data availability to the processor, and returns to the Idle state.

CHAPTER 5

SOFTWARE DESIGN

5.1 Software Overview

The eSC-V processor operates in a bare-metal, Machine-mode execution environment. Because the system is designed for resource-constrained, high-security embedded microcontrollers, there is no underlying operating system, hypervisor, or runtime monitor. Instead, a single firmware image compiles directly to machine instructions and runs with full privilege on the physical hardware.

To execute software on the core, application code written in C is compiled, linked, and formatted into a hexadecimal memory file. During FPGA synthesis, this initialization file is hardcoded into the dual-port unified memory unit, integrating the software directly into the synthesized bitstream. Upon release from the hardware power-on-reset sequence, the processor begins fetching instructions from the hardwired boot address `0x80000000`.

The software execution life cycle is divided into three distinct runtime phases:

1. **Low-Level Initialization (Bootstrapping):** The CPU first executes the low-level assembly startup script (`crt0.S`). This script configures the CPU's internal architecture, initializes the stack and frame pointers, clears the uninitialized global variables in the BSS memory segment, and sets up exception handlers. During this phase, the startup script also configures the core's security configuration registers to enable the hardware landing pad checking (`Zicfilp`) and configures the physical memory protection (PMP) address limits to lock down the secure shadow stack (`Smcfiss` / `Smpmpind`).
2. **Application Execution:** Once the runtime environment is configured, control is transferred to the C standard library entry point, which calls the C application's main function. C applications execute standard algorithms, manipulate data structures, and interface with the physical world using memory-mapped I/O (MMIO) registers. Standard C library functions, such as character output and memory allocations, are supported by routing calls through system-level stubs to the UART transceiver and local heap managers.
3. **Exception and Trap Handling:** If the CPU encounters an illegal instruction, an unaligned memory access, a PMP privilege violation, or a hardware security fault (such as a JOP branch targeting a non-landing-pad instruction or an ROP exploit corrupting a return address), it suspends normal execution. The hardware immediately redirects control to the registered trap vector handler. This handler disables the pipeline's security configuration registers to prevent nested traps, captures the architectural registers and exception cause codes, and prints a crash dump to the UART interface.

5.2 Bare-Metal Bootstrapping & Memory Layout

Establishing a reliable startup sequence and a strictly structured memory layout is critical for bare-metal execution. In the eSC-V system, this is managed by the custom linker script `link.ld` and the initialization assembly code `crt0.S`. Together, they coordinate the physical mapping of memory regions and configure the CPU registers before C code execution begins.

5.2.1 Memory Map & Section Allocation

The linker script `link.ld` partitions the target 32KB RAM space (originating at address `0x80000000` and ending at `0x80008000`) to isolate code, variables, stacks, and heap space:

- **Executable Code (.text):** Placed at the beginning of the RAM address space (`0x80000000`). It uses the `KEEP` directive to lock the initialization assembly section (`.text.init`) containing the entry symbol (`_start`) at the boot address, ensuring it executes immediately upon release from reset.
- **Read-Only Data (.rodata):** Placed immediately after the code segment to house constants, string literals, and static tables.
- **Initialized Variables (.data / .sdata):** Placed next, containing variables initialized to non-zero values.
- **Uninitialized Variables (.bss / .sbss):** Positioned after the initialized variables. The linker defines two tracking symbols—`_bss_start` and `_bss_end`—to mark the boundaries of this block, which must be cleared by the startup code.
- **Heap Space (.heap_start):** Allocated immediately after the BSS segment, growing upward toward the stack boundaries.
- **System Stack Allocation:** The program stack is allocated at the very top of the 32KB RAM space (`0x80008000`), growing downward with a fixed size of 2KB. The top of this region is marked by the `_stack_top` symbol, and the bottom is marked by `_stack_bottom` (`0x80007800`).
- **Hardware-Isolated Shadow Stack:** To support the `Smcfiss` security extension, a dedicated 1KB shadow stack is allocated immediately below the program stack, with its top boundary (`_shadow_stack_top`) starting at the bottom of the program stack (`0x80007800`) and its bottom boundary (`_shadow_stack_bottom`) set to `0x80007400`.
- **Linker Build Assertion:** To prevent memory corruption, the linker script includes a build-time assertion:

```
ASSERT(_end <= _shadow_stack_bottom, "Error: Heap collides with shadow stack")
```

This assertion halts compilation if the combined size of the code, variables, and heap grows upward and overlaps with the shadow stack region.

5.2.2 Startup Assembly Runtime

When the CPU is released from reset, it jumps to the entry symbol `_start` in `crt0.S`, executing the following bootstrapping sequence:

1. Trap Vector Registration

Before performing any memory operations, the startup code registers the exception handler. It loads

the address of the trap handler (`_trap_handler`) and writes it to the machine trap vector base address register (`mtvec`, CSR `0x305`). This ensures that if any hardware fault or security violation occurs during bootstrapping, the CPU traps to a safe handler.

2. Stack and Frame Pointer Initialization

The startup code loads the `_stack_top` symbol into the stack pointer register (`sp`) and copies it to the frame pointer register (`s0`), establishing the stack boundaries for subsequent function calls.

3. Clearing the BSS Segment

To satisfy C language specifications, all uninitialized global and static variables must be cleared to zero before execution. The startup code runs a loop from `_bss_start` to `_bss_end`, writing zero to each memory location.

4. Enabling Zicfilp Landing Pads

If compiled with forward-edge CFI protections enabled (`__ENABLE_ZICFILP__`), the startup code writes 1 to the machine security configuration register (`mseccfg`, CSR `0x747`). This activates landing pad enforcement in the processor's Execute stage.

5. Enabling Smcfiss & Smpmpind Shadow Stack

If compiled with shadow stack protections enabled (`__ENABLE_SMCFISS__`), the startup code configures physical memory protection (PMP) registers to protect the shadow stack region:

- It shifts the shadow stack bottom address (`_shadow_stack_bottom`) and top address (`_shadow_stack_top`) right by 2 bits to match the 4-byte PMP address granularity, writing them to PMP address registers 0 (`0x3B0`) and 1 (`0x3B1`).
- It selects PMP region 1 by writing 1 to the indirect selector register (`miselect`, CSR `0x350`).
- It writes the configuration value `0x8000000A` to the indirect configuration register (`miereg2`, CSR `0x352`). This configures the memory region between PMP region 0 and region 1 to Top of Range (TOR) mode with write-only permissions. Asserting bit 31 designates it as a secure shadow stack region.
- It writes the shadow stack top address (`_shadow_stack_top`) to the shadow stack pointer register (`ssp`, CSR `0x011`).
- It reads the `mseccfg` register, sets bit 2 to enable shadow stack checking, and writes it back.

6. Transferring Control to C

Once initialization is complete, the startup code calls the main function. If the C program completes, the processor enters an infinite loop.

7. Trap and Exception Handling

If an exception or security violation occurs, the CPU jumps to the `_trap_handler` symbol. The handler first writes zero to the `mseccfg` register, disabling security checks to prevent nested traps during debugging. It then reads the exception cause register (`mcause`, CSR `0x342`), the exception program counter

(mepc, CSR 0x341), and the trap value register (mtval, CSR 0x343). These values, along with the saved frame pointer, are passed to the C-based `print_crash_dump` routine, which prints the CPU state and registers to the UART interface.

5.3 C Standard Library Integration & System Call Interface

To facilitate rapid software development using high-level C code, the eSC-V system incorporates Newlib, a lightweight, open-source C standard library (`libc`) tailored specifically for bare-metal and embedded systems. Integrating a standard library allows application software to use standard calls like `printf` and `malloc`. However, in a bare-metal environment, Newlib does not have access to an underlying operating system kernel. Consequently, the software must implement low-level system call stubs (`syscalls.c`) to bridge the gap between Newlib's standard functions and the memory-mapped physical peripherals.

5.3.1 Low-Level System Call Stubs

The stubs defined in `syscalls.c` redirect standard library operations to the eSC-V hardware interface:

Standard Output Redirection (`_write`): Standard library output functions (such as `printf` and `puts`) eventually call the low-level `_write` stub. The `_write` stub checks if the file descriptor matches standard output (`STDOUT_FILENO`) or standard error (`STDERR_FILENO`). If it does, the stub loops through the character buffer, writing each byte sequentially to the physical transmitter using the UART driver.

Standard Input Redirection (`_read`): Standard input functions (such as `scanf` or `getchar`) call the `_read` stub. The stub validates that the request is targeting standard input (`STDIN_FILENO`) and performs a polling loop, calling the UART driver to retrieve incoming bytes. The read loop continues until the requested buffer length is filled or a carriage return/newline character is detected.

Dynamic Heap Management (`_sbrk`): Dynamic memory allocation functions (such as `malloc` and `free`) rely on the `_sbrk` stub to grow and shrink the heap. The stub references the linker symbols `_end` (marking the end of the BSS segment and the start of the heap) and `_stack_bottom` (marking the lower limit of the stack region). When a memory allocation is requested, `_sbrk` verifies that the updated heap pointer will not collide with the stack space.

If a collision is detected, the stub sets the error code to `ENOMEM` (Out of Memory) and returns an error. Otherwise, it increments the heap boundary, protecting the stack and shadow stack from heap overflows.

Peripheral Identification (`_fstat` / `_stat`): When Newlib queries input/output streams, the `_fstat` and `_stat` stubs intercept the call. They set the device mode to a character special file (`S_IFCHR`), informing Newlib that the file descriptor represents an interactive character terminal (the UART serial interface) rather than a block storage file.

Operating System Stubs: Stubs for features not supported by the bare-metal environment—such as process creation (`_fork`, `_execve`), signal handling (`_kill`), file system access (`_open`, `_unlink`),

and timing (`_times`)—return error codes (e.g. `ENOSYS` or `EBADF`) to satisfy the linker while preventing invalid execution. If the C program exits, the `_exit` stub enters an infinite loop containing NOP instructions, keeping the CPU in a safe state.

5.3.2 Upstream Contribution to the Newlib Library

A major contribution of this project is an upstream patch merged into the official global Newlib codebase, authored by Arjun Salunkhe. This patch resolved a critical compatibility issue between hardware-enforced Control-Flow Integrity (CFI) and standard library implementations.

1. The Security Compatibility Challenge

To defend against Jump-Oriented Programming (JOP) attacks, the eSC-V core implements the RISC-V `Zicfilp` extension. Under `Zicfilp`, the hardware requires that every target of an indirect call or jump must begin with a landing pad (`lpad`) instruction. If the CPU redirects execution to an address that does not begin with `lpad`, it raises a landing pad exception and halts execution.

During software compilation, the custom GNU GCC toolchain was configured with `Zicfilp` support, allowing the compiler to automatically insert `lpad` instructions at the entry points of all compiled C functions. However, when executing C programs linked against standard Newlib libraries on the physical eSC-V core, the CPU consistently trapped and crashed immediately upon executing library functions.

2. Debugging and Root Cause Analysis

Debugging the failure required tracing the exact instruction sequence leading to the trap. Analysis revealed that the issue was not caused by a hardware design defect or compiler error, but rather by standard library optimizations.

For performance-critical memory and string operations (such as `memset`, `memcpy`, `memmove`, and `strlen`), Newlib uses handwritten, highly-optimized assembly files rather than standard C implementations. Because these assembly files are written manually, they bypass the GCC C compiler entirely. Consequently, the compiler was unable to insert the required `lpad` instructions at their entry points. When the C compiler generated indirect jumps to these optimized assembly routines, the eSC-V hardware strictly enforced the `Zicfilp` policy, detected the missing `lpad` instructions, and halted execution.

3. The Resolution and Upstream Integration

The fix involved manually adding the `lpad 0` instruction to the entry points of the optimized assembly routines in the RISC-V subdirectory of Newlib codebase. Once the library was recompiled with these changes, standard C library functions executed on the `Zicfilp`-enabled core without triggering traps.

Rather than keeping this fix as a local workaround, the project leader submitted the patch upstream to the Newlib mailing list. Following code review and feedback from RISC-V toolchain maintainer Kito Cheng, the patch was accepted and merged into the official Newlib codebase. This contribution ensures that out-of-the-box Newlib distributions are compatible with `Zicfilp`-hardened RISC-V hardware, representing a significant open-source contribution from this final-year design project.

5.4 Hardware Peripheral & Debug Drivers

To abstract the physical hardware and provide diagnostic capabilities, the eSC-V software stack includes dedicated drivers for peripheral communication and real-time debugging. The peripheral drivers interface with the memory-mapped I/O (MMIO) registers, while the debug drivers extract internal CPU states and print them to a terminal-based dashboard.

5.4.1 UART Transceiver Driver (`uart.c` & `uart.h`)

The UART driver (`uart.c`) provides the software interface for serial I/O. It maps registers to the MMIO space starting at base address `0x00000000` using volatile byte pointers to prevent compiler optimizations:

Registers and Offsets: The data register (`UART_DR`) is mapped to offset `0x00`, and the status register (`UART_STATUS`) is mapped to offset `0x04`. Status checks extract the transmitter ready flag (`UART_TX_READY`) from bit 0 and the receiver data valid flag (`UART_RX_VALID`) from bit 1.

Character Transmission (`uart_putc`): When transmitting a character, the function polls the transmitter ready bit in a loop until the UART hardware indicates it is idle. The character is then written to the data register. If a newline character (`\n`) is sent, the function automatically transmits a carriage return (`\r`) as well, ensuring correct cursor alignment on standard serial terminals.

Character Reception (`uart_getc`): The reception routine operates non-blockingly. It first inspects the receiver valid bit in the status register. If the bit is zero, the function returns immediately. If the bit is set, it reads the received byte from the data register, writes zero to the status register to clear the hardware valid flag, and returns the byte.

String Output (`uart_puts`): Translates null-terminated strings into sequential calls to the character transmission function.

5.4.2 Hardware Diagnostic & Debug Monitor (`debug.c`, `core_debug.S` & `debug.h`)

To help developers verify hardware execution and trace CFI traps, the software stack includes a real-time diagnostic monitor. This monitor creates a split-screen dashboard on the serial terminal, separating application output from internal hardware states.

Low-Level Register Extraction (`core_debug.S`): Written in assembly, this module implements routines to extract the processor's register state. The function `_dump_gprs` saves the values of all 32 general-purpose registers (`x1–x31`) to a memory buffer. In parallel, `_dump_csrs` reads and saves the values of 8 critical CSRs: the trap vector address (`mtvec`), security configuration (`mseccfg`), cycle counter (`mcycle`), instruction counter (`minstret`), shadow stack pointer (`ssp`), scratch register (`mscratch`), PMP configuration (`pmpcfg0`), and PMP address 0 (`pmpaddr0`). Consistent with the core's security rules, these assembly routines are CFI-hardened, utilizing landing pads (`lpad0`) at their entry points and protecting return addresses using shadow stack push (`ssppush`) and verification (`sspopchk`) instructions.

5.5 Ported Applications & Validation Suite

To demonstrate the functionality of the eSC-V SoC and validate its hardware-enforced Control-Flow Integrity (CFI) protections under real-world conditions, a suite of user applications and security validation tests was developed. These applications run directly on the bare-metal environment, utilizing the UART peripheral for I/O and display.

5.5.1 Ported Interactive Applications

Two open-source terminal-based games were ported to the eSC-V platform. Porting these applications demonstrated the core's ability to handle complex game loops, coordinate rendering, and receive asynchronous user inputs over serial connections:

Pong-C: Ported from the open-source repository `pbogre/pong.c`, this game implements full-duplex Pong over the UART interface. The game logic manages ball physics, collision boundaries, paddle coordinates, and player scores. To display the game, the rendering routine generates standard ANSI escape sequences to clear the terminal, draw the boundaries, and update paddle and ball positions in real time. Player movements are captured by polling the UART receiver for keyboard inputs.

ASCII-Tetris: Ported from the open-source repository `Gregwar/ASCII-Tetris`, this game implements a full-featured Tetris game. It maintains a grid matrix representing falling tetromino blocks, detects collision boundaries during block rotations, clears completed rows, and increments scores. The game uses ANSI terminal escapes to draw the falling blocks. In parallel, it calls the debugging dashboard API on each game tick, drawing a live trace of the general-purpose registers, security CSR states, and active call stack on the right side of the screen. This provides a visual demonstration of the hardware executing complex software under load.

5.5.2 Functional and Security Validation Suite

The verification suite (tests) is divided into functional library tests and security exploit simulations:

Libc Validation (`libc`): A test suite that exercises C standard library integrations. It verifies dynamic memory allocation via `malloc` and `free` (validating the heap safety checks in the `_sbrk` system call), string parsing, and formatted I/O operations, ensuring that the Newlib library functions correctly on the custom CPU.

CFI Security Validation (`main.c`): A security test program designed to simulate software vulnerabilities and verify that the hardware blocks exploits. It simulates two classic attack vectors:

1. **Return-Oriented Programming (ROP) Simulation:** The program implements a vulnerable function with an unchecked stack buffer. It prompts the user to input a payload via UART that overflows the local buffer, overwriting the return address on the stack. When the shadow stack (`Smcfiss`) is disabled, returning from the function redirects execution to an unauthorized target function, demonstrating arbitrary code execution. When the shadow

stack is enabled, the Writeback stage detects that the return address popped from memory does not match the value on the hardware shadow stack, blocking execution and trapping the CPU.

2. **Jump-Oriented Programming (JOP) Simulation:** The program declares a structure containing a character buffer followed by a function pointer. It prompts the user to input a payload that overflows the buffer, overwriting the function pointer with the address of the target function. When the program calls the function pointer (an indirect jump), and landing pads (Zicfilp) are disabled, the jump succeeds. When landing pads are enabled, the Execute stage checks the target address. Since the target function does not begin with a valid landing pad (1pad) instruction, the CPU raises a landing pad fault and halts execution.

5.6 Web-Based SoC Demonstration Interface

To make the eSC-V system more accessible to evaluators and observers, the software stack includes an interactive web-based demonstration interface (demo). This interface consists of a Vite-based frontend web application and a lightweight Python serial server backend.

5.6.1 Rationale for a Graphical Interface

Hardware security and processor micro-architectures are highly abstract concepts. Presenting a live demonstration solely within a Linux terminal using raw serial console logs can be difficult for non-specialist observers to follow or fully appreciate. The web-based demonstration interface was created to bridge this gap, translating command-line logs into a graphical dashboard that illustrates execution flows, memory states, and security traps.

5.6.2 Visual Parsing & Live State Display

The demonstration interface connects the physical hardware to the web application:

Python Serial Backend: The Python server runs on the host PC, establishing a serial connection to the physical Tang Primer FPGA board or simulation console via a USB UART interface. It reads the raw data stream sent from the core's serial output and forwards it to the web application.

Diagnostic Parsing: The web application parses the diagnostic packet strings emitted by the CPU's debug library. It extracts the values of the 32 general-purpose registers, the 8 active CSR configurations, and the active call stack frames.

Frontend Dashboard: The Vite frontend displays these parsed values in a real-time graphical interface. As users play Pong or Tetris, the web page displays the registers and stack frames updating dynamically. During security tests, the interface displays how a buffer overflow modifies memory, and highlights the landing pad or shadow stack trap catching the violation.

Accuracy Trade-off: While the displayed stack and register files are not 100% architecturally identical to the application's native state (as executing the debug print routine itself pushes variables

onto the stack and uses temporary registers), this representation is sufficient to demonstrate the hardware's internal operations and security policies.

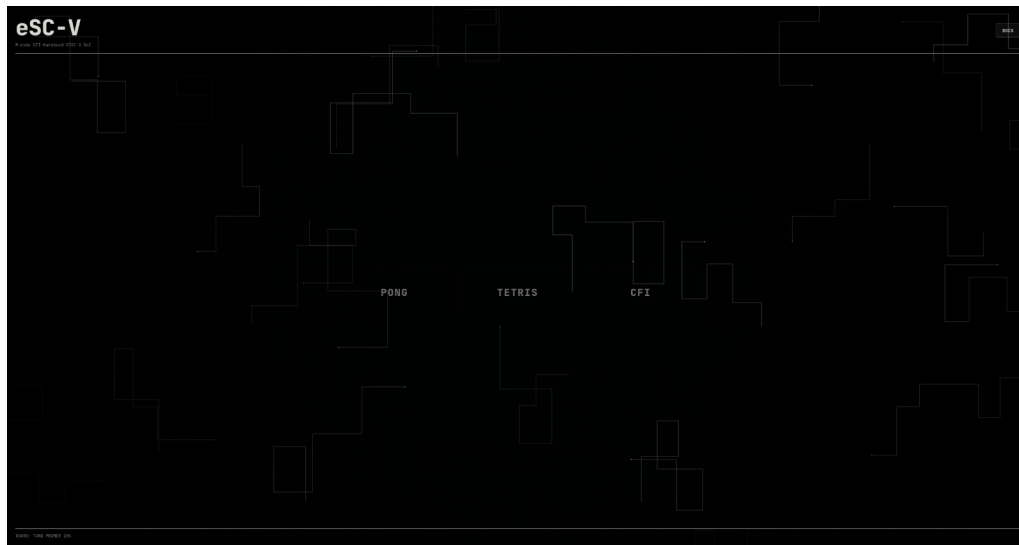


Figure 3: Web-Based SoC Demonstration Interface Home Page.



Figure 4: Interactive ASCII-Tetris Game Layout with Live Architectural Metrics.



Figure 5: Full-Duplex Pong-C Application Dashboard over Serial Stream.

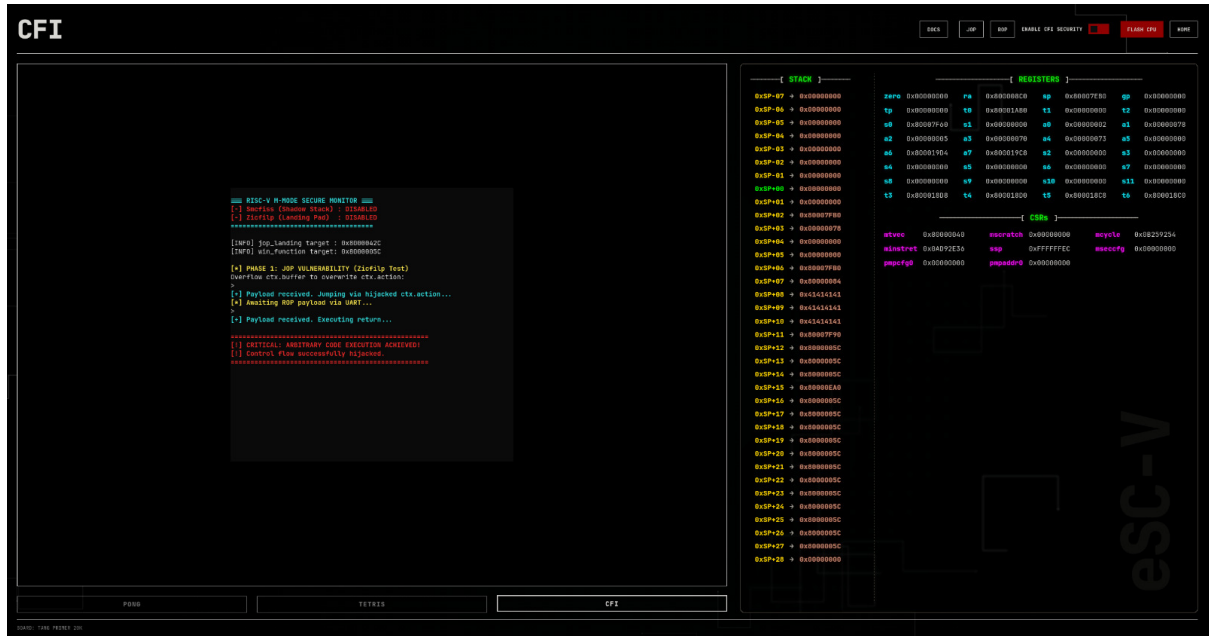


Figure 6: Unsafe CFI Simulation (Hardware Protections Disabled — Attack Vector Successfully Exploited).

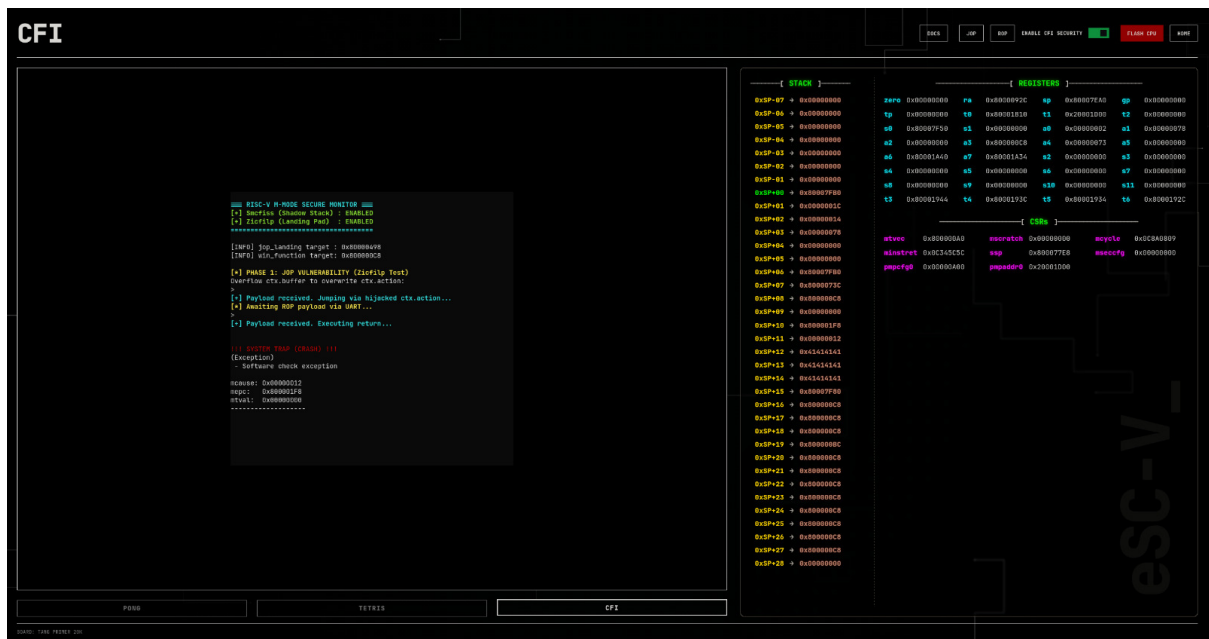


Figure 7: Safe CFI Simulation (Hardware Protections Enabled — Exploit Defeated via Precise System Trap).

CHAPTER 6

METHODOLOGY

6.1 Methodology Overview

The methodology for securing the eSC-V System-on-Chip focuses on enforcing Control-Flow Integrity (CFI) at the hardware level. In standard software execution, control-flow instructions (such as jumps, calls, and returns) dictate the path of code execution. Under normal conditions, these transfers follow a strict, developer-defined Control Flow Graph (CFG). However, software vulnerabilities—such as stack buffer overflows or unvalidated pointer writes—allow malicious actors to hijack these transfers, redirecting execution to arbitrary locations. This class of exploits, known as Control-Flow Hijacking, is the foundation for Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP) attacks.

To counter these threats, the eSC-V processor implements a dual-enforcement CFI architecture:

- **Forward-Edge CFI (Zicfilp):** Protects indirect calls and jumps (forward-edge transitions) by enforcing the Zicfilp extension. The hardware requires that any indirect jump must target a valid, compiler-inserted landing pad instruction containing a matching label. This blocks JOP attacks, which rely on jumping to arbitrary instructions.
- **Backward-Edge CFI (Smcfiss & Smpmpind):** Protects function returns (backward-edge transitions) by implementing the Smcfiss and Smpmpind extensions. The hardware maintains a duplicate stack of return addresses in a secure, write-protected memory region (the shadow stack). Upon function return, the CPU compares the return address on the program stack with the value saved on the shadow stack, trapping the processor if a mismatch is detected. This blocks ROP attacks, which rely on overwriting return addresses on the stack.

By executing these checks combinationally in the Execute stage and verifying them at instruction commit in the Writeback stage, the eSC-V core protects bare-metal applications from ROP and JOP exploits without requiring software-level runtime monitors.

6.2 Control-Flow Hijacking Mechanics

Control-flow hijacking is a class of software exploits where an attacker overwrites pointer values to divert the processor's execution path away from the program's compiled Control Flow Graph (CFG). Modern operating systems and embedded environments implement basic memory protections—such as Write-XOR-Execute (W^X)—which mark data regions (like the stack and heap) as non-executable. This blocks traditional shellcode injection attacks. To bypass these restrictions, modern exploits use code-reuse attacks, such as Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP). These attacks execute arbitrary logic by chaining together existing instructions in the compiled binary.

6.2.1 Return-Oriented Programming (ROP)

Return-Oriented Programming (ROP) targets backward-edge control-flow transitions, which represent the return paths of functions.

Vulnerability Exploitation: In the standard C call stack layout, local variables, saved frame pointers, and function return addresses are stored sequentially on the same stack. If a program accepts user

input into a stack buffer without checking its boundary limits, the input can overflow the buffer. This allows an attacker to overwrite the adjacent saved return address on the stack.

The Return Instruction: When a function finishes execution, it executes a return instruction (such as `jr ra` or `ret` in RISC-V). The CPU reads the return address from the stack, loads it into the Program Counter, and continues execution from that location.

Chaining Gadgets: Instead of injecting malicious code, the attacker scans the program's compiled binary (including linked standard libraries) to identify short instruction sequences, known as "gadgets," that end in a return instruction. By carefully structuring the overflow payload on the stack, the attacker replaces the original return address with the address of the first gadget.

Execution Flow: When the active function returns, it jumps to the first gadget. The first gadget executes a few operations (such as loading a value into a register) and then executes a return instruction. This return pops the next address off the stack, jumping to the second gadget. By repeating this process, the attacker can chain multiple gadgets together to execute arbitrary operations (such as modifying system registers or writing to protected memory) using only existing, authorized code.

6.2.2 Jump-Oriented Programming (JOP)

Jump-Oriented Programming (JOP) targets forward-edge control-flow transitions, which represent function calls and indirect jumps.

Vulnerability Exploitation: In C and C++ programs, indirect calls (calling a function via a function pointer) and indirect jumps (jumping to a register address) are common in callback functions, virtual method tables (vtables), and dynamic dispatch systems. If an attacker corrupts a function pointer in memory (using a heap overflow, use-after-free, or buffer overflow), they can redirect the target address of the indirect call.

Chaining JOP Gadgets: JOP attacks do not rely on return instructions or stack modifications. Instead, they chain together gadgets that end in register-indirect jumps or register-indirect calls (such as `jr rs1` or `jalr rs1`).

The Dispatcher Gadget: Because JOP cannot use return instructions to pop the next gadget address off the stack, it uses a "dispatcher gadget." This is a short loop that reads a list of gadget addresses from memory, increments an index register, and jumps indirectly to the next address. By controlling the index register and the list of gadget addresses in memory, the attacker forces the dispatcher gadget to jump to the target gadgets sequentially. JOP attacks bypass stack-based ROP protections entirely because they do not modify the stack pointer or execute return instructions.

6.3 Hardware-Enforced Mitigations

To protect bare-metal applications from Return-Oriented Programming (ROP) and Jump-Oriented Programming (JOP) attacks, the eSC-V core implements hardware-enforced Control-Flow Integrity (CFI) protections. These mitigations secure both backward-edge transitions (function returns) and forward-edge transitions (indirect calls and jumps), blocking exploits before they can execute unauthorized code.

6.3.1 Backward-Edge Hardening: Stopping ROP via Smcfiss & Smpmpind

The core defends against ROP exploits by implementing a hardware-managed shadow stack under the Smcfiss extension, isolated from the standard software environment using the Smpmpind extension.

Dual Stack Storage: The core maintains two stack regions: the standard program stack in RAM (which stores local variables, frame pointers, and return addresses) and a secure shadow stack (which stores only return addresses). The shadow stack pointer is stored in the dedicated, private control and status register ssp (CSR csr_unit.vhd).

Call/Return Verification: When a function call executes, the hardware automatically pushes the return address onto both the program stack and the shadow stack. When the function returns, the CPU pops the return address from the program stack and the expected return address from the shadow stack. In the Writeback stage, the hardware compares these two values. If the return address on the program stack has been overwritten by a buffer overflow, the mismatch is detected, and the CPU raises a shadow stack fault trap, halting execution before the hijacked address can be loaded into the Program Counter.

Smpmpind Memory Protection: To prevent attackers from overwriting the shadow stack itself, the core uses physical memory protection (PMP) registers. PMP region 1 is configured in Top of Range (TOR) mode with write-only permissions. By asserting a custom configuration bit in the PMP register, the region is designated as the shadow stack memory space. The PMP unit enforces access controls:

1. **Standard Software Access:** Any standard store instruction (like SB, SH, or SW) attempting to write to the shadow stack region triggers a PMP store access fault, trapping the processor.
2. **Hardware Shadow Stack Access:** Only dedicated hardware shadow stack instructions (such as a call push) are allowed to write to this memory space.
3. **Access Isolation:** If a hardware shadow stack instruction attempts to access a standard memory region outside the designated shadow stack boundaries, the PMP unit blocks the write. This prevents shadow stack operations from being redirected to overwrite standard program data.

6.3.2 Forward-Edge Hardening: Stopping JOP via Zicfilp

The core defends against JOP exploits by implementing state-based landing pad verification under the Zicfilp extension.

Expected Landing Pad (ELP) State Tracker: The core tracks the expected landing pad state using an internal state bit. When the processor executes an indirect jump or indirect call (excluding standard return-register jumps to $x1/ra$, $x5/t0$, or $x7/t2$), the Execute stage sets the expected landing pad state to active, signaling this condition to the Fetch stage.

Target Validation: On the subsequent clock cycle, the CPU fetches the instruction at the jump target address. If the expected landing pad state is active, the target instruction must be a valid landing pad (lpad) instruction. If the target is not a landing pad, the Execute stage immediately raises a landing pad fault trap, halting execution.

Label Matching: To prevent an attacker from jumping to a landing pad at the entry point of a different, unauthorized function, Zicfilp enforces label matching. The compiler assigns a unique immediate label to each indirect jump and its target landing pad. During execution, the expected label is loaded into register $x7$ ($t2$). When the indirect jump lands, the ALU subtracts the landing pad instruction's immediate label from the value in register $x7$ ($t2$). If the result is zero, the labels match, and execution continues. If the labels do not match, a landing pad fault is raised, trapping the CPU. This restricts indirect jumps to authorized target entry points.

CHAPTER 7

REQUIREMENTS

7.1 Requirements Overview

The development, synthesis, and verification of the eSC-V system present a tightly coupled hardware-software co-design challenge. Because the core objective of the eSC-V architecture is to pioneer a secure, hardware-enforced Control-Flow Integrity (CFI) framework using the RISC-V Zicfilp and Zicfiss extensions, the engineering requirements must span both physical hardware targets and an intricate software compilation and simulation stack. To ensure the long-term viability, auditability, and accessibility of this project, the formulation of these requirements is guided by three foundational engineering principles.

7.1.1 Foundational Engineering Principles

The architectural choices for both the hardware platform and the supporting software development stack are governed by a strict set of design constraints intended to maximize transparency, environmental stability, and security alignment:

Absolute Commitment to a FOSS Ecosystem: Traditional digital design workflows rely heavily on proprietary Electronic Design Automation (EDA) suites and vendor-locked FPGA toolchains. These closed ecosystems introduce opaque black-box optimizations, restrictive licensing, and severe cross-platform compatibility barriers. The eSC-V project deliberately rejects this paradigm. A primary requirement for both the hardware synthesis target and the software pipeline is an entirely Free and Open-Source Software (FOSS) architecture. By prioritizing reverse-engineered bitstream generation tools and open-source synthesis engines, the project ensures that the entire silicon-to-bitstream pipeline remains fully transparent, auditable, and customizable. This philosophy directly dictates the selection of hardware components that support open-source toolchains.

Hermetic Environmental Determinism: Hardware-software co-design pipelines are notorious for environment drift. A minor divergence in a host system's library path, compiler version, or environment variables can lead to non-deterministic synthesis results or subtle execution bugs in the target software. To eliminate these cross-platform discrepancies, the eSC-V project mandates a completely declarative and isolated development environment. Every tool—from the VHDL simulator to the custom cross-compiler—must be explicitly versioned, dependency-locked, and reproducible across any host machine. This requirement is met by managing the entire development infrastructure via Nix flakes, guaranteeing that any developer can instantiate an identical bitstream and binary compilation environment instantaneously.

Co-Optimization for Advanced Security Extensions: Unlike standard baseline RISC-V implementations, the eSC-V architecture incorporates cutting-edge, hardware-enforced security primitives that demand explicit cooperation from the compiler. The toolchain must not only support standard RISC-V instructions but also recognize and correctly emit specialized security structures like landing pads (lpad) and shadow stack operations. Consequently, standard off-the-shelf toolchains are insufficient. The requirements dictate a custom-built compilation framework engineered specifically to target the `rv32i_zicsr_zicfilp_zicfiss` architecture string, tightly in-

tegrated with the underlying low-level runtime libraries (Newlib) to ensure end-to-end security enforcement.

7.2 Hardware Requirements

To ground the eSC-V custom security processor in a stable physical medium, the prototyping pipeline requires a balanced evaluation target and an open-source compatible communication bridge. The hardware execution footprint is defined by a resource-dense FPGA platform paired with an on-hand JTAG programming interface to maintain absolute open-source transparency and operational economy.

7.2.1 Tang Primer 20k FPGA Development Platform

The main computing matrix for the eSC-V system-on-chip synthesis is the Sipeed Tang Primer 20k core board, a compact hardware architecture designed around Gowin semicustom gate arrays.



Figure 8: Sipeed Tang Primer 20k Diagram

Detailed Technical Specifications: The core platform is anchored by the Gowin GW2A-LV18PG256C8/I7 silicon. This chip yields 20,736 4-input Look-Up Tables (LUT4), 15,552 digital Flip-Flops (FF), 41,472 bits of high-speed Shadow SRAM, and 828 Kb of Block SRAM (B-SRAM) distributed uniformly across 46 independent blocks. Vector processing and specialized arithmetic operations are supported by 48 embedded 18x18 hardware multipliers alongside 4 internal phase-locked loops (PLLs) for multi-rail system clock alignment. System memory capabilities are extended by an onboard 128 MB DDR3 SDRAM module and 32 Mbit of SPI NOR Flash storage.

System Usage & Integration: Within the eSC-V framework, the Tang Primer 20k holds the completely compiled VHDL core architecture. The programmable logic fabric implements the processor's decoding registers, multi-stage pipelines, Arithmetic Logic Units (ALUs), and physical memory protection (PMP) structures. Memory blocks are partition-mapped to establish an isolated shadow stack run space, while physical I/O boundaries map the physical UART lines out to host machines for telemetry readouts.

Project Motivation: The fundamental engineering justification for this board is its mature compatibility with Project Apicula—a comprehensive, reverse-engineered open-source bitstream compila-

tion toolset. This enables compilation through Yosys and nextpnr-himbaechel, removing dependencies on proprietary vendor IDE licenses and keeping the entire project open source. Furthermore, the 20k LUT footprint offers an ideal resource density, providing sufficient room to house the custom hardware control-flow extensions (Zicfilp and Zicfiss) without inducing routing congestion or massive propagation delays.

Economic Profile & Cost: The development kit provides mid-range logical density at a minimal financial footprint, retailing between 25.00 and 35.00 USD. This highly accessible pricing model fulfills the baseline project goals of economic efficiency and simple laboratory replication.

7.2.2 ESP-PROG JTAG Debugger Interface

Deploying the compiled bitstream onto the physical FPGA fabric requires an external hardware programmer capable of translating software calls into JTAG logic streams.

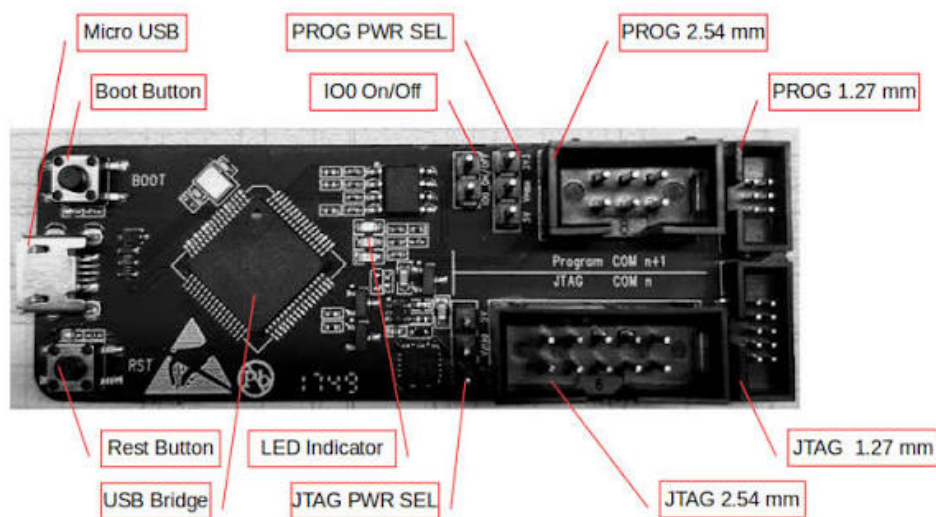


Figure 9: ESP-PROG Diagram

Detailed Technical Specifications: The ESP-PROG is an open-hardware configuration tool powered by Espressif's implementation of the FTDI FT2232HL dual-channel USB-to-UART/JTAG bridge IC. It supports automated data transfers up to 30 Mbps over a standard JTAG loop and operates at variable 3.3V and 5.0V signal logic thresholds. The hardware splits its tasks between two channels: Channel A controls synchronous JTAG lines (TCK, TMS, TDI, TDO), while Channel B routes an independent asynchronous UART serial bus.

System Usage & Integration: The ESP-PROG serves as the physical link between the developer's workstation and the JTAG configuration pins located on the FPGA board. During execution, the open-source utility openFPGALoader utilizes the ESP-PROG interface to program the compiled binary bitstream configuration (.fs structure) straight into the volatile SRAM matrix or the SPI flash of the target core.

Project Motivation: The selection of the ESP-PROG was a pragmatic engineering decision made because a functional module was already available on hand in lab inventory. Utilizing this device bypassed procurement delays and eliminated the need for closed-source, vendor-specific programmers. Because the underlying FT2232HL interface chip is universally supported by open-source flashing tools, it integrates into the declarative Nix flake environment without requiring proprietary binary wrappers or closed kernel modules.

Economic Profile & Cost: Although a standalone ESP-PROG module sells within a budget range of 10.00 to 15.00 USD, its effective cost to this project iteration is 0.00 USD due to asset recovery and laboratory recycling. This reinforces the project's goal of resource optimization and strict cost control.

7.3 Software Requirements

To support the security extensions and maintain full transparency, the eSC-V software ecosystem is entirely built on open-source projects. Managed by Nix flakes, this setup provides a deterministic, reproducible environment that avoids version drift across host systems. The software stack is split into two primary pipelines: the hardware synthesis and Electronic Design Automation (EDA) suite, and the custom cross-compilation toolchain.



Figure 10: Nix Flakes

7.3.1 Open-Source Hardware Synthesis and EDA Suite

The physical translation of VHDL source code into an executable FPGA bitstream relies on a completely open-source synthesis and implementation chain. This setup replaces proprietary, vendor-locked IDEs with modular command-line tools.

GHDL (VHDL Simulation Ingestion): GHDL is an open-source analyzer, compiler, and simulator for the VHDL language. Unlike typical interpretive simulators, GHDL compiles VHDL files directly into native host machine code. In this project, GHDL serves a dual role: it provides high-fidelity, event-driven simulation for logic verification, and it acts as the front-end parser that feeds the hardware design into the synthesis pipeline.

Yosys RTL Synthesis Suite & yosys-ghdl: Yosys is a feature-rich framework for Register-Transfer Level (RTL) synthesis. It converts behavioral HDL descriptions into technology-mapped netlists. Because Yosys natively focuses on Verilog, the architecture incorporates the yosys-ghdl plugin. This integration allows Yosys to directly ingest complex VHDL-93/2008 design files parsed by GHDL, optimizing the hardware control logic and mapping it into generic internal gates.

Nextpnr Place-and-Route Engine: Nextpnr is a portable, vendor-neutral place-and-route tool designed for FPGA architectures. It takes the structural netlist generated by Yosys and maps it to the physical layout of the target FPGA. For this project, Nextpnr runs via the himbaechel backend, which handles Gowin architectures. It dynamically places logic cells and routes interconnects while satisfying timing constraints.

Project Apicula (apycula): Project Apicula is a reverse-engineered, open-source documentation toolset specifically for Gowin FPGAs. It maps the proprietary bitstream formats of Gowin chips. apycula takes the post-place-and-route database from Nextpnr and generates the final, physical configuration bitstream (.fs file) required by the Tang Primer 20k core.

openFPGALoader: This lightweight, universal utility flashes bitstreams onto various FPGA fabrics. It handles JTAG protocols directly from the command line, communicating with the FT2232HL dual-channel bridge chip on the ESP-PROG hardware. This utility handles transferring the compiled .fs structure into either the volatile SRAM matrix for active debugging or the onboard SPI NOR Flash for non-volatile storage.

GTKWave: GTKWave is a fully featured, wave viewer that reads standard Value Change Dump (.vcd) files generated during GHDL simulation. It provides a visual timeline of internal register transitions, pipeline states, and bus transactions, making it an essential tool for debugging complex core operations before actual hardware deployment.

7.3.2 Custom Compilation Toolchain and Build Systems

Because the eSC-V core uses advanced security primitives like Zicfilp (Forward-Edge CFI via Landing Pads) and Zicfiss (Backward-Edge CFI via Shadow Stacks), standard pre-compiled RISC-V toolchains cannot be used. Instead, the project uses a custom-built GNU GCC cross-compiler tailored to emit these specialized instructions.

Custom GNU GCC Toolchain Configuration: The system cross-compiler is built from source using highly restrictive configuration flags to guarantee that the generated code aligns with the processor's physical hardware constraints:

```
./configure --prefix=$PWD/install \
            --with-arch=rv32i_zicsr_zicfilp_zicfiss \
            --with-abi=ilp32 \
            --disable-multilib
```

The `--with-arch` flag configures the compiler to target a base RV32I core extended by Control and Status Registers (Zicsr), Landing Pads (Zicfilp), and Shadow Stacks (Zicfiss). Disabling multilib (`--disable-multilib`) locks the output to a single, optimized target runtime environment, preventing linking errors.

Target Compilation Parameters: To enforce security across all compiled binaries, target compilation uses strict parameters passed via the build system:

```
make -j14 CFLAGS_FOR_TARGET="-O2 -fcf-protection=full \
    -march=rv32i_zicsr_zicfilp_zicfiss -mabi=ilp32"
```

The flag `-fcf-protection=full` forces the compiler to automatically inject instrumentation points. This means it inserts `lpad` instructions at the start of all C function entry points and handles shadow stack mechanics for function returns. The `-O2` optimization level balances execution speed with predictable assembly output.

GnuMake (gnumake): GnuMake serves as the foundational build automation engine for this project. It coordinates multi-stage assembly, handles file dependencies, and manages compiler passes. This automation ensures that low-level C code, handwritten system call stubs (`syscalls.c`), and architecture-specific Newlib libraries compile and link into accurate Executable and Linkable Format (`.elf`) binaries.

Hermetic Flake Packaging: To simplify development, this custom toolchain was pre-built, tarballed, and hosted as a dedicated release within the eSC-V repository. The Nix flake fetches this release and applies an automated patching hook (`autoPatchelfHook`). This dynamically links host system libraries (like `glibc`, `gmp`, and `mpfr`) straight to the toolchain's binaries. This setup provides an instant, isolated, and identical development environment on any host machine running Nix.

CHAPTER 8

VERIFICATION

To verify that the custom eSC-V processor core conforms to the official RISC-V architecture specifications, the hardware design was integrated with the RISC-V Architectural Compliance Test Framework (RISCOF). Verification is a critical milestone in processor design, ensuring that compiler-generated code executes with predictable and compliant results. The test suite was executed within the `riscov` directory.

8.1 Overview of the RISCOF Framework

RISCOF is the standardized framework used by the RISC-V International organization to validate processor compliance. It operates by executing a suite of architectural assembly tests (`riscv-arch-test`) designed to exercise every corner of the instruction set.

Rather than checking CPU registers during execution, the framework relies on memory signatures. During simulation, the test program writes its execution results to a designated signature memory region. Upon test completion, the signature memory is dumped to a file. RISCOF then compares this signature against the signature generated by a gold-standard reference model—specifically the Sail RISC-V formal model (the official mathematical definition of the ISA) and the Spike ISA simulator. If the signatures match, the processor is validated as compliant for that instruction.

8.2 Hardware Verification Testbench & SoC Debug Port Integration

To support signature capture, the top-level System-on-Chip wrapper in `soc.vhd` exposes conditionally compiled debug signals. These signals expose internal memory operations—including the target memory address, write data, write enable, and byte-lane enables—directly to the testbench.

The compliance testbench (`tb_soc_riscov.vhd`) integrates these signals:

- **Shadow Memory Copy:** The testbench maintains a shadow memory array that mirrors all data writes from the CPU. By using the byte-lane enables, the testbench updates the shadow memory byte-by-byte, ensuring accurate tracking of byte, halfword, and word store instructions.
- **Test Termination:** Compliance tests signal completion by writing a value to a magic memory address (mapped to the `tohost` symbol in the program's symbol table). The testbench monitors the debug address bus, and when a write to this termination address is detected, it interrupts simulation, dumps the signature memory range (from `begin_signature` to `end_signature`) to a signature output file, and terminates the simulation.
- **Timeout Handling:** If a test enters an infinite loop or hangs due to a hardware defect, a cycle counter in the testbench triggers a timeout after 200,000 clock cycles. It dumps a partial signature and terminates the simulation to prevent test suite hangs.

8.3 Custom Python Plugin & Test Orchestration

To automate verification, a custom Python plugin (`riscov_eSC_V.py`) was developed to interface the RISCOF framework with the GHDL VHDL simulator:

- **VHDL Compilation:** The plugin compiles the core and SoC VHDL source files sequentially using the GHDL compiler, generating the compiled work library.

- **Test Compilation:** For each architectural test, the plugin compiles the assembly source code into an ELF binary using the RISC-V GCC cross-compiler.
- **Symbol Extraction:** The plugin runs the RISC-V nm utility on the ELF binary to extract the exact byte offsets of the begin_signature, end_signature, and tohost symbols.
- **HEX File Conversion:** It converts the compiled ELF binary into a flat hexadecimal memory file (code.hex) to initialize the SoC Block RAM model.
- **Simulation Execution:** The plugin executes GHDL, passing the extracted symbol offsets as VHDL generics to the testbench. GHDL runs the simulation and generates the raw signature output, which the plugin formats and returns to the RISCOF framework.

8.4 Verification Coverage & Compliance Results

The eSC-V core was tested against the official RV32I base integer instruction set. The verification suite executed tests covering:

- **Arithmetic and Logic:** Addition, subtraction, logical operations, and register-to-immediate calculations.
- **Shifts and Comparisons:** Logical and arithmetic shifts, and signed/unsigned comparisons.
- **Control Flow:** Conditional branches, direct jumps, and register-indirect jumps.
- **Memory Access:** Byte, halfword, and word loads and stores, verifying byte-lane alignment and PMP overrides.
- **System Registers:** Zicsr status register read, write, and bitwise set/clear instructions.

The eSC-V processor achieved 100% test coverage over the RV32I and Zicsr compliance suites. All generated signatures matched the outputs of the Sail formal model and the Spike simulator with zero mismatches, proving that the hardware design is fully compliant with the RISC-V architectural specifications.

CHAPTER 9

RESULTS

9.1 Synthesis and Performance Overhead Results

To quantify the architectural cost of integrating hardware-enforced Control-Flow Integrity (CFI) protections, a comparative synthesis and benchmark analysis was performed. The baseline eSC-V core (devoid of security extensions) was evaluated against the fully hardened CFI-enabled core implementing the Zicfilp, Smcfiss, and Smpmpind specifications. Both configurations were synthesized for the target Tang Primer 20k FPGA using the open-source Yosys and nextpnr toolchain. The results reveal predictable trade-offs across physical logic utilization, timing performance, and software code density.



Figure 11: Results Graph

9.1.1 Hardware Resource Utilization and Timing Analysis

The implementation of real-time security validation logic fundamentally requires extra physical hardware resources and modifies the core's critical routing paths.

Area and Logic Footprint (LUT4 Utilization): The baseline processor utilizes 7,527 LUTs, whereas the CFI-enabled core expands to 8,066 LUTs, representing a +6.68% area overhead. This minor increase is directly attributed to the extra combinational logic required to track the Expected Landing Pad (ELP) state machine, compute label mismatches in the Execute stage, and manage the validation paths. Given the robust security guarantees provided, this physical footprint increase is negligible and well within the capacity of the target FPGA fabric.

Total Registers (Sequential Storage Impact): Sequential cell utilization exhibits minimal variation, moving from 2,461 cells in the baseline core to 2,469 cells in the secure variant—a trivial +0.33% increase. The addition of exactly 8 register cells reflects the instantiation of the dedicated hardware shadow stack pointer (ssp) Control and Status Register (CSR) and supplementary configuration latches within the Physical Memory Protection (PMP) unit. The remaining core control registers remain unaffected, demonstrating a highly optimized sequential design.

Maximum Operational Frequency (F_{max}): Physical timing analysis shows a drop in maximum operational frequency from 58 MHz (baseline) down to 54 MHz (CFI-enabled), indicating a -6.89% reduction in maximum clock speed. This frequency degradation is primarily driven by the propagation delays introduced by the PMP bounds-checking logic. Verifying memory bounds on every shadow stack push and pop introduces extra levels of logic along the processor's critical path. However, because this represents an unoptimized alpha layout, restructuring the address-matching arithmetic and tightening pipeline stage boundaries will allow heavier optimization in subsequent hardware iterations to reclaim this performance loss.

9.1.2 Software Binary Footprint Evaluation

The impact of hardware-enforced security mechanisms extends past the silicon surface into the compiled machine code density.

Binary Size Expansion: Compiling a standard suite of bare-metal application binaries results in an increase from a 19 KB baseline to 22.8 KB for the CFI-hardened version, which constitutes a significant +19.82% memory expansion. This overhead is caused by the compiler instrumentation required to secure forward and backward edges.

Toolchain Maturity and Alpha Status: The notable code size inflation is explicitly tied to the current experimental state of the compilation framework. Because the underlying GCC tooling supporting the specialized `rv32i_zicsr_zicfilp_zicfiss` architecture string is still in its alpha stages—featuring direct custom code contributions from this project—optimization passes for automatic landing pad (lpad) layout and shadow stack manipulation are not yet fully matured. The toolchain frequently emits conservative instruction sequences to ensure strict safety compliance rather than code minimization. As open-source development continues and upstream library optimizations stabilize, this compilation overhead is expected to drop substantially.

CHAPTER 10

CONCLUSION

10.1 Conclusion

The eSC-V project successfully demonstrates the design, verification, and physical realization of a secure, hardware-enforced Control-Flow Integrity (CFI) framework optimized for low-power, microcontroller-class embedded systems. By architecting a custom 32-bit RV32I system-on-chip (SoC) from scratch in VHDL, this design delivers a resilient, bare-metal execution environment engineered to neutralize complex Code-Reuse Attacks (CRAs) directly within the hardware pipeline.

10.1.1 Summary of Architectural and Security Achievements

The implementation successfully bridges the gap between low-level hardware security primitives and a functional bare-metal software runtime through several verified milestones:

Processor and SoC Architecture: The foundational core features a standard 5-stage pipelined execution matrix. Pipeline hazards are handled dynamically via a dedicated Hazard Detection Unit that resolves load-use dependencies, alongside a combinational Forwarding Unit that eliminates data hazards. The surrounding SoC incorporates a dual-port memory controller mapped directly to the FPGA's internal Block RAM (BRAM) and a memory-mapped UART transceiver for full-duplex serial communication.

Hardware-Enforced CFI Hardening: The processor pipeline integrates the ratified RISC-V Zicfilp extension to secure forward-edge transitions using landing pad verification. Backward-edge transitions are hardened by implementing the draft Smcfiss and Smpmpind extensions, which manage an isolated hardware shadow stack. In the absence of a virtual Memory Management Unit (MMU), the shadow stack is protected against unauthorized software modification via strict physical memory protection (PMP) address validation. Any detected control-flow violations trigger an immediate two-stage pipeline flush, isolating the core and trapping the processor into a secure state.

Formal Verification and FPGA Deployment: Architectural correctness was validated using the official RISCOF compliance framework, achieving 100% compliance across the RV32I base instruction set when evaluated against the Sail formal model and the Spike reference simulator. The entire open-source hardware synthesis pipeline successfully targeted the Gowin silicon on a Sipeed Tang Primer 20K FPGA, maintaining stable execution at a clock frequency of 54 MHz.

Bare-Metal Software Stack: Environment configuration and cross-compilation tools were managed deterministically via Nix Flakes. The software ecosystem successfully executes complex embedded C code utilizing a custom assembly bootstrap sequence (`crt0.S`) and a low-level system call interface stubbed into the Newlib C library (`syscalls.c`). This complete hardware-software co-design was verified by running interactive, real-time applications, including Pong-C and ASCII-Tetris, over a local serial loop.

10.1.2 Security Paradigm and Resource Efficiency

As modern embedded microcontrollers face an expanding threat landscape, they remain uniquely vulnerable due to the lack of power-heavy memory management hardware like MMUs. The eSC-V architecture proves that highly robust control-flow security can be offloaded entirely to the processor pipeline, intercepting JOP and ROP exploits at the internal bus layer before malicious instructions can compromise system integrity.

Crucially, these guarantees are achieved while incurring a minimal 6.68% hardware logic overhead. This efficiency establishes that hardware-enforced CFI is a practical, cost-effective, and scalable security strategy for resource-constrained, real-world embedded deployments.

10.1.3 Open-Source Ecosystem and Upstream Contributions

Beyond internal design achievements, the project delivered a meaningful contribution to the broader open-source RISC-V ecosystem. During the integration of the Newlib runtime library with the custom compiler's automated Zicfilp protections, critical runtime faults were traced to handwritten assembly routines that bypassed standard C compiler instrumentation.

To resolve this compatibility barrier, the project leader, Arjun Salunkhe, engineered and submitted an architectural patch to the official global Newlib mailing list. Following a peer review process with core toolchain maintainers, the patch was merged into the master Newlib codebase. This contribution ensures that future out-of-the-box Newlib distributions natively support Zicfilp-hardened RISC-V targets worldwide.

10.1.4 Future Research and Development Directions

While the initial eSC-V prototype is fully functional and formally verified, iterative advancements will focus on enhancing the SoC's scalability and execution throughput:

Dynamic Bootloading Infrastructure: Future revisions will replace the hardcoded BRAM memory initialization files with a dynamic bootloader. This secondary boot routine will allow developers to stream compiled binary applications directly into volatile memory over the UART interface post-startup, eliminating the need to re-synthesize the FPGA bitstream for software updates.

Standardized Bus and Peripheral Extensions: To support advanced application deployment, the SoC interconnect will be expanded to incorporate a standardized open-source system bus interface, such as Wishbone. This extension will facilitate modular integration with external SPI flash memory controllers, DDR RAM interfaces, and a broader array of secondary communication peripherals.

Pipeline Performance Optimization: To reclaim the frequency and cycle overheads introduced by real-time security checking, the core execution stages will undergo targeted timing optimization. This includes the development of an integrated hardware Branch Prediction Unit (BPU) and local instruction/data cache controllers to minimize pipeline stalls and enhance overall instruction throughput.

CHAPTER 11

REFERENCES

11.1 REFERENCES

1. T. Mishra, T. Chantem, and R. Gerdes, “Survey of Control-flow Integrity Techniques for Real-time Embedded Systems,” *ACM Trans. Embed. Comput. Syst.*, vol. 21, pp. 1–32, 2022. [Online]. Available: <https://doi.org/10.1145/3538275>
2. L. Szekeres, M. Payer, T. Wei, and D. Song, “SoK: Eternal War in Memory,” in *2013 IEEE Symp. Secur. Privacy (SP)*, Berkeley, CA, USA, 2013, pp. 48–62. [Online]. Available: <https://doi.org/10.1109/SP.2013.13>
3. R. de Clercq et al., “SOFIA: Software and control flow integrity architecture,” in *2016 Design, Automation & Test in Europe Conf. & Exhibition (DATE)*, 2016. [Online]. Available: <https://ieeexplore.ieee.org/document/7459489>
4. A. De, A. Basu, S. Ghosh, and T. Jaeger, “FIXER: Flow Integrity Extensions for Embedded RISC-V,” in *2019 Design, Automation & Test in Europe Conf. & Exhibition (DATE)*, 2019, pp. 348–353. [Online]. Available: <https://doi.org/10.23919/DATE.2019.8714980>
5. N. Maunero, P. Prinetto, G. Roascio, et al., “A FPGA-based Control-Flow Integrity Solution for Securing Bare-Metal Embedded Systems,” in *2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*, 2020, pp. 1–10. [Online]. Available: <https://doi.org/10.1109/DTIS48698.2020.9081314>
6. R. J. Walls, N. F. Brown, T. Le Baron, et al., “Control-flow integrity for real-time embedded systems,” in *31st Euromicro Conf. Real-Time Syst. (ECRTS)*, 2019. [Online]. Available: <https://www.ll.mit.edu/r-d/publications/control-flow-integrity-real-time-embedded-systems>
7. RISC-V International. (2026). “Ratified extensions,” RISC-V Tech Hub. [Online]. Available: <https://riscv.org/specifications/ratified/>
8. ethycS0. (2026). “eSC-V: RV32I SoC in VHDL with Hardware-enforced CFI,” GitHub. [Online]. Available: <https://github.com/ethycS0/eSC-V>
9. RISC-V International. (2026). “Control-flow integrity (CFI),” RISC-V Ratified Specifications Library. [Online]. Available: <https://docs.riscv.org/reference/isa/priv/priv-cfi.html>
10. Ved-rivos. (2026). “RISCV-ISA-MANUAL Extension for Machine-mode Shadow Stack,” GitHub. [Online]. Available: <https://github.com/ved-rivos/riscv-isa-manual>
11. S. Manoni et al., “CVA6-CFI: A first glance at RISC-V control-flow integrity extensions,” 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.04991>
12. RISC-V International. (2024, Jun.). “RISC-V ratifies additional specifications in June board meeting.” [Online]. Available: <https://lists.riscv.org/g/tech-announce/message/474>
13. GCC GNU Compiler Collection. (2024, Nov.). “[PATCH 2/4] RISC-V: Add Zicfilp ISA extension.” [Online]. Available: <https://gcc.gnu.org/pipermail/gcc-patches/2024-November/668919.html>

14. YosysHQ. (2026). “Project Apicula: Bitstream documentation for Gowin FPGAs,” GitHub. [Online]. Available: <https://github.com/YosysHQ/apicula>
15. Sipeed Wiki. (2026). “Tang Primer 20K.” [Online]. Available: <https://wiki.sipeed.com/hardware/en/tang/tang-primer-20k/primer-20k.html>
16. Hackaday. (2022, Jun. 12). “An open toolchain for Sipeed Tang Nano FPGAs.” [Online]. Available: <https://hackaday.com/2022/06/12/an-open-toolchain-for-sipeed-tang-nano-fpgas/>
17. M. A. Elshiemy and A. Soltan, “Comprehensive Compliance Framework for RISC-V Architectures using the RISCOF Test Platform,” in *2025 IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2025, pp. 1–5. [Online]. Available: <https://doi.org/10.1109/ISCAS56072.2025.11043314>
18. RISCOF. (2026). “RISCOF 1.23.4 Documentation.” [Online]. Available: <https://riscov.readthedocs.io/en/latest/intro.html>
19. RISC-V Non-ISA. (2026). “riscv-arch-test repository,” GitHub. [Online]. Available: <https://github.com/riscv-non-isa/riscv-arch-test>

CHAPTER 12

PROJECT ACHIEVEMENTS

12.1 Peer-Reviewed Publications & Research Contributions

This section outlines the peer-reviewed research papers successfully published, presented, and contributed to the scientific community during the development lifecycle of the eSC-V project.

12.1.1 Main Research Paper

Title: *eSC-V: Hardware-Enforced CFI Protection for Machine-Mode RISC-V SoC*

Authors: Salunkhe, A., Vaidya, K., Bhosale, A., Bansode, S., & Shinde, S.

Journal Publication: Published in the *IEEE Xplore*, 2026.

Conference Presentation: Presented at the *1st International Conference on Innovation & Growth in Next-Gen Intelligent Technology & Engineering (IGNITE)*, 2026.

12.1.2 Systematization of Knowledge (SoK) Paper

Title: *SoK: Hardware-Enforced Control-Flow Integrity Across the Privilege Spectrum*

Authors: Salunkhe, A., Vaidya, K., Bhosale, A., Bansode, S., & Shinde, S.

Journal Publication: Published in the *Cureus Journal of Computer Science*, 2026.

Conference Presentation: Presented at the *International Conference on Emerging Techniques in Computational Intelligence (ICETCI)*, 2026.

12.2 National Level Project Competition (NLPC) 2026

The eSC-V project was showcased in the National Level Project Competition (NLPC) organized by IETE. The project was recognized for its innovative approach to hardware-enforced bare-metal security. The certificates of participation are included below.

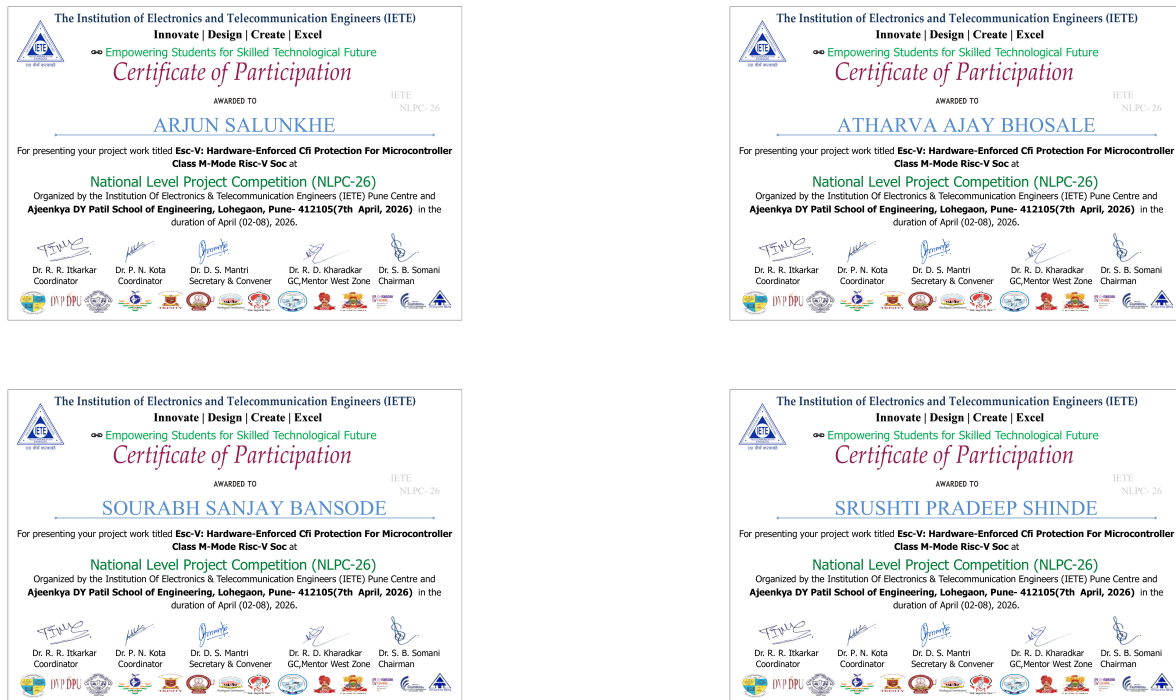


Figure 12: Participation certificates from NLPC 2026 organized by IETE.

12.3 Sponsorship: We acknowledge PROWIZ SYSTEMS for their technical input during the project.

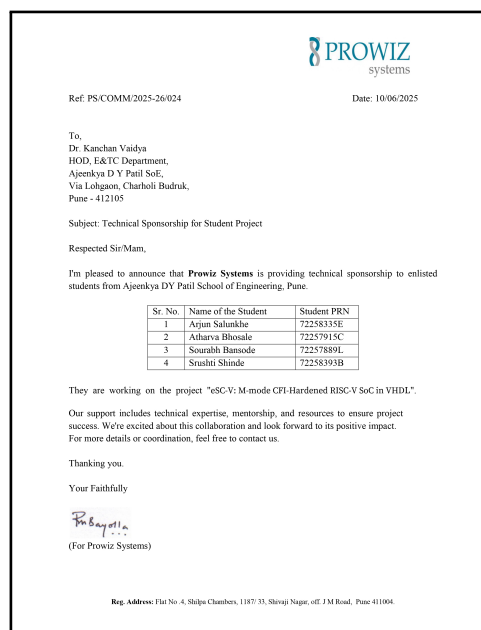


Figure 13: Sponsorship letter.